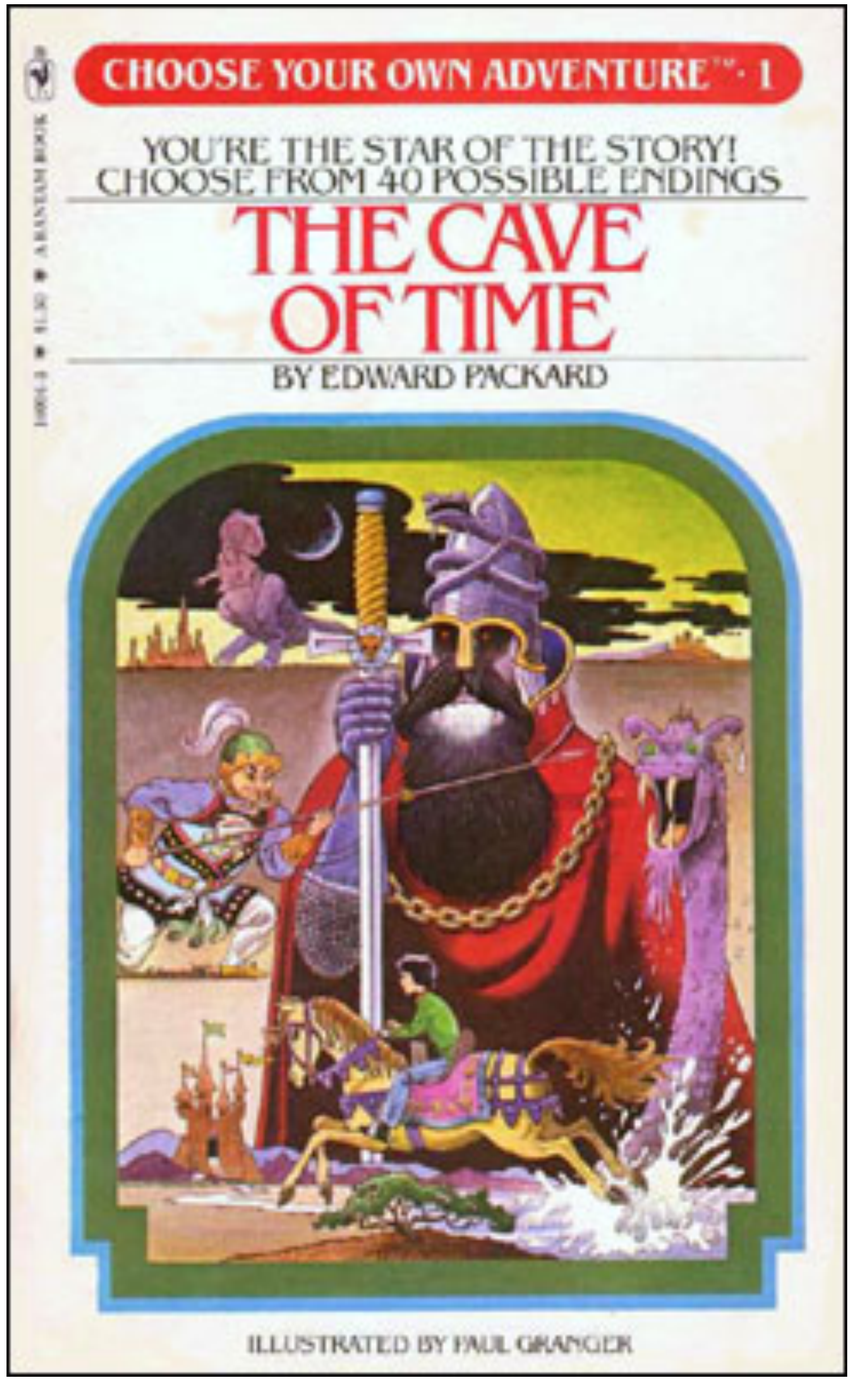
Decision trees, k-nearest neighbor, model selection

10-701 Introduction to Machine Learning
Geoff Gordon

(thanks to Matt Gormley and Henry Chai for some content)

Decision trees



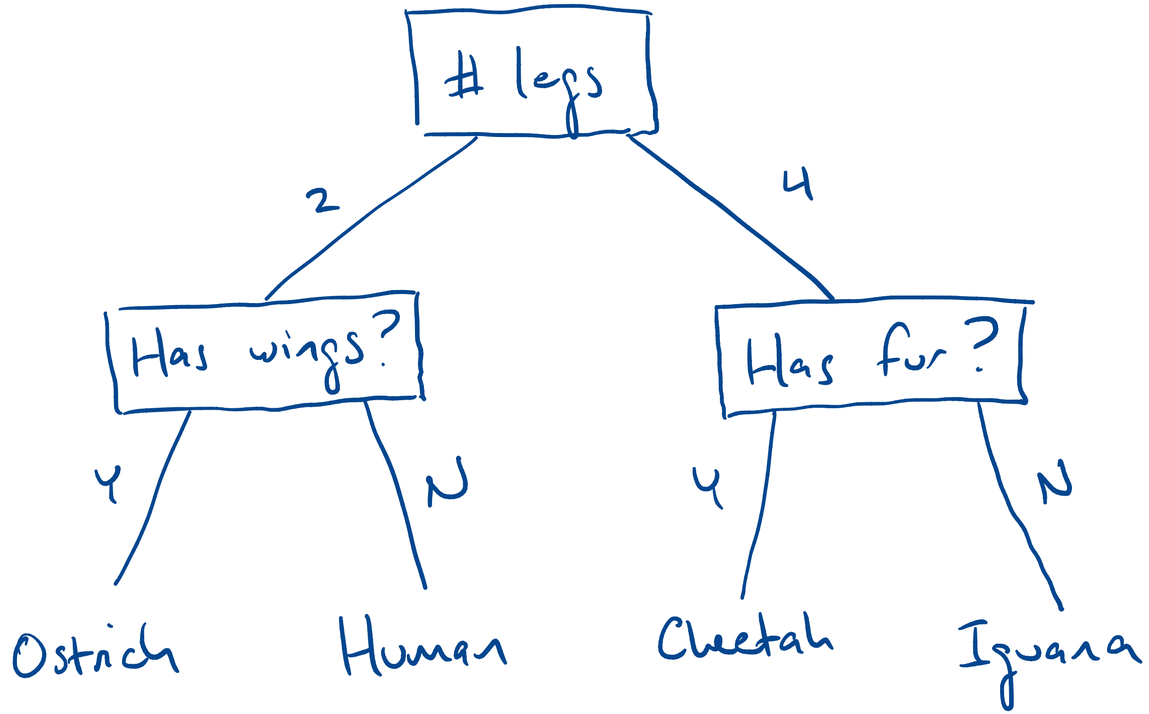
Choose your own adventure!

If you decide to open the creepy sarcophagus, turn to p63

If you decide to back slowly out of the room, turn to p17

If you decide to set up camp for the night, turn to p42

Decision trees



- A question in each node, an answer on each edge
  - answers mutually exclusive and exhaustive
- Each question splits the data into two (or more) pieces
- When we reach a leaf, it makes a prediction

Supplemental reading: Hastie & Tibshirani, ch 9

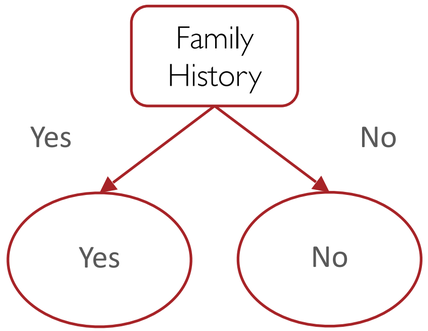
Decision tree example

features

labels

training data points

Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes

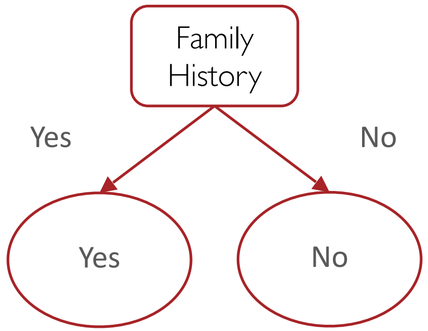


Decision tree example

features | labels | predictions

training data points

Family History	Resting Blood Pressure	Cholesterol	Heart Disease?	Model output
Yes	Low	Normal	No	Yes
No	Medium	Normal	No	No
No	Low	Abnormal	Yes	No
Yes	Medium	Normal	Yes	Yes
Yes	High	Abnormal	Yes	Yes

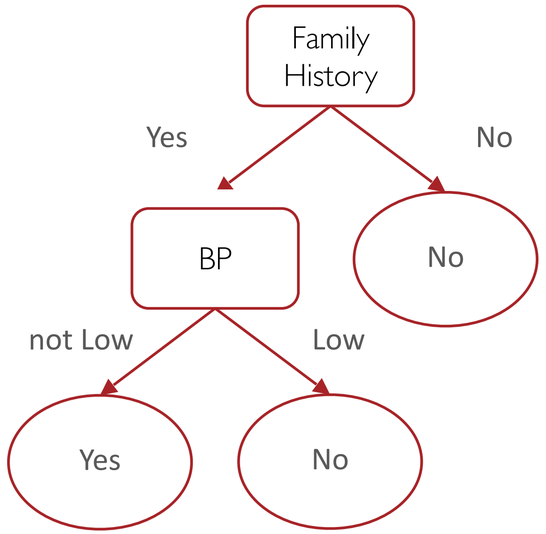


Decision tree example

features | labels | predictions

training data points

Family History	Resting Blood Pressure	Cholesterol	Heart Disease?	Model output
Yes	Low	Normal	No	Yes
No	Medium	Normal	No	No
No	Low	Abnormal	Yes	No
Yes	Medium	Normal	Yes	Yes
Yes	High	Abnormal	Yes	Yes

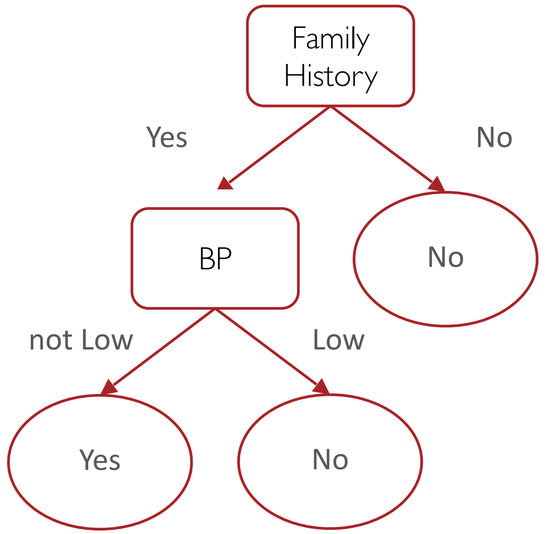


Decision tree example

features labels predictions

training data points

Family History	Resting Blood Pressure	Cholesterol	Heart Disease?	Model output
Yes	Low	Normal	No	No
No	Medium	Normal	No	No
No	Low	Abnormal	Yes	No
Yes	Medium	Normal	Yes	Yes
Yes	High	Abnormal	Yes	Yes

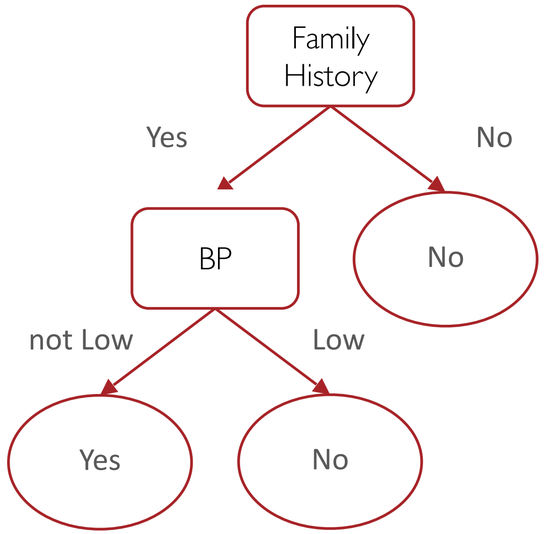


Decision tree example

features | labels | predictions

training data points

Family History	Resting Blood Pressure	Cholesterol	Heart Disease?	Model output
Yes	Low	Normal	No	No
No	Medium	Normal	No	No
No	Low	Abnormal	Yes	No
Yes	Medium	Normal	Yes	Yes
Yes	High	Abnormal	Yes	Yes

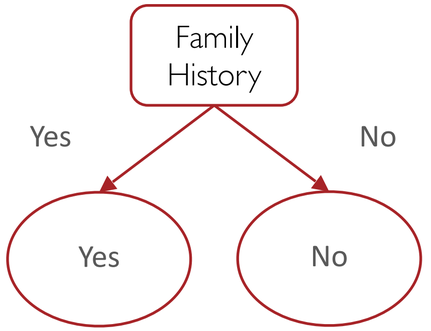


if just one split: "decision stump"

Splitting criterion

Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes

- How do we choose which features to split on?
  - example criterion: minimize training error



→ 2 errors total

Yes	Low	Normal	No
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes

1 error

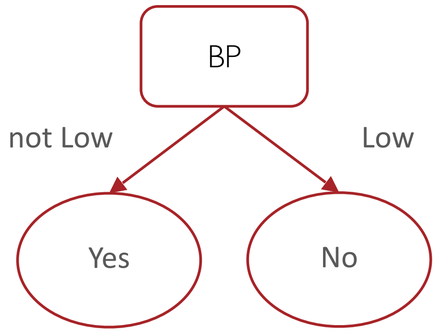
No	Medium	Normal	No
No	Low	Abnormal	Yes

1 error

Splitting criterion

Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes

- How do we choose which features to split on?
  - example criterion: minimize training error



→ 2 errors total

No	Medium	Normal	No
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes

1 error

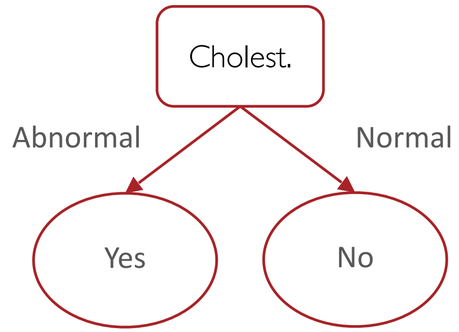
Yes	Low	Normal	No
No	Low	Abnormal	Yes

1 error

Splitting criterion

Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes

- How do we choose which features to split on?
  - example criterion: minimize training error



→ 1 error total, ***best!***

No	Low	Abnormal	Yes
Yes	High	Abnormal	Yes

0 errors

Yes	Low	Normal	No
No	Medium	Normal	No
Yes	Medium	Normal	Yes

1 error

Decision tree learning

- We could ask for the decision tree that minimizes error on training set (subject to constraints, e.g., max depth)
  - but this is a difficult combinatorial optimization problem
- Instead, typically use a heuristic search
  - find a good tree but maybe not the absolute minimum
- Most common: greedy top-down search
  - pick the split at the root according to some ***split criterion***
  - recursively learn left and right children, using the appropriate subset of the data for each
  - when we stop (e.g., threshold on depth or number of examples in subtree) use *majority* classifier on the leaf

Decision Tree — Training

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

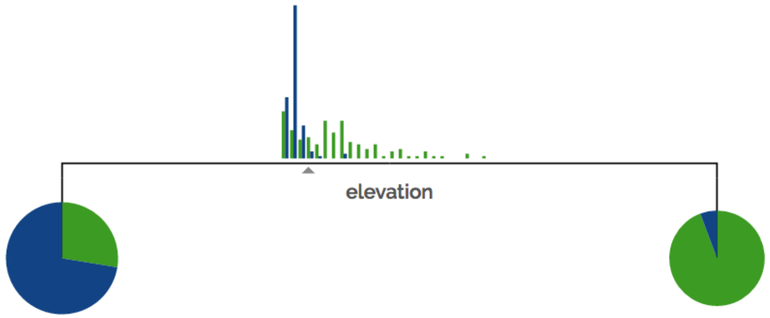
- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

slide credit: Ameet Talwalkar

Decision Tree — Training

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

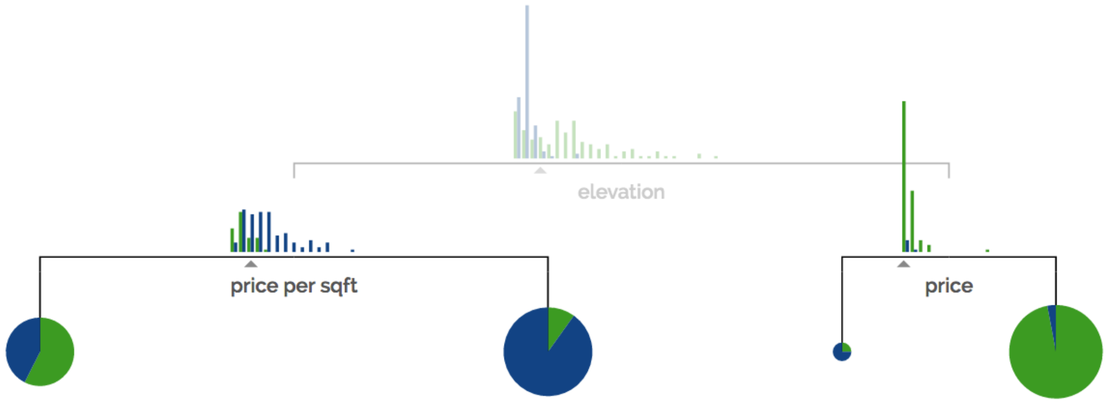


slide credit: Ameet Talwalkar

Decision Tree — Training

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

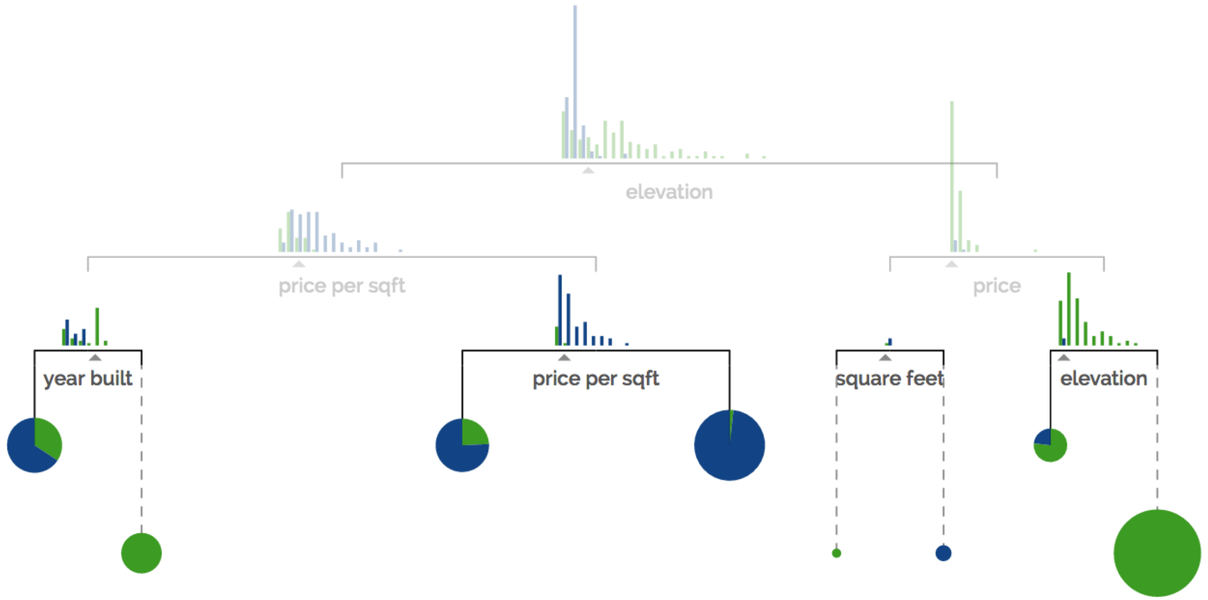


slide credit: Ameet Talwalkar

Decision Tree — Training

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

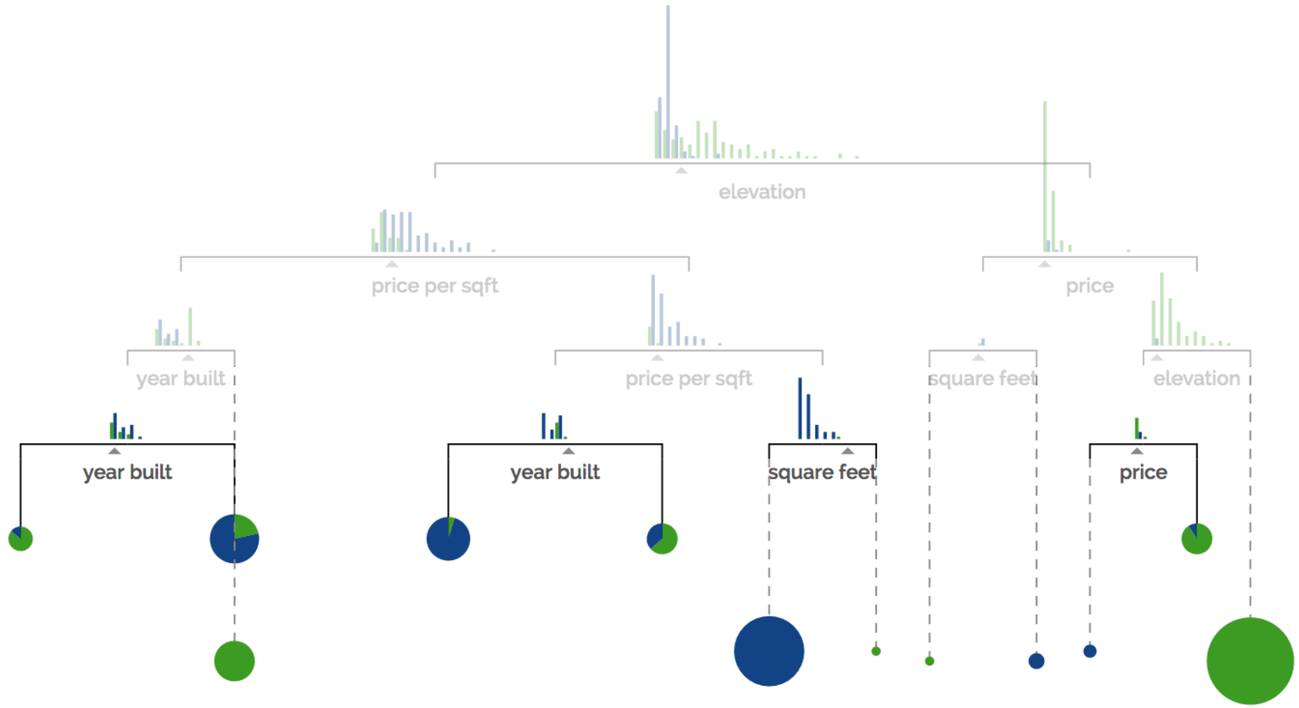


slide credit: Ameet Talwalkar

Decision Tree — Training

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

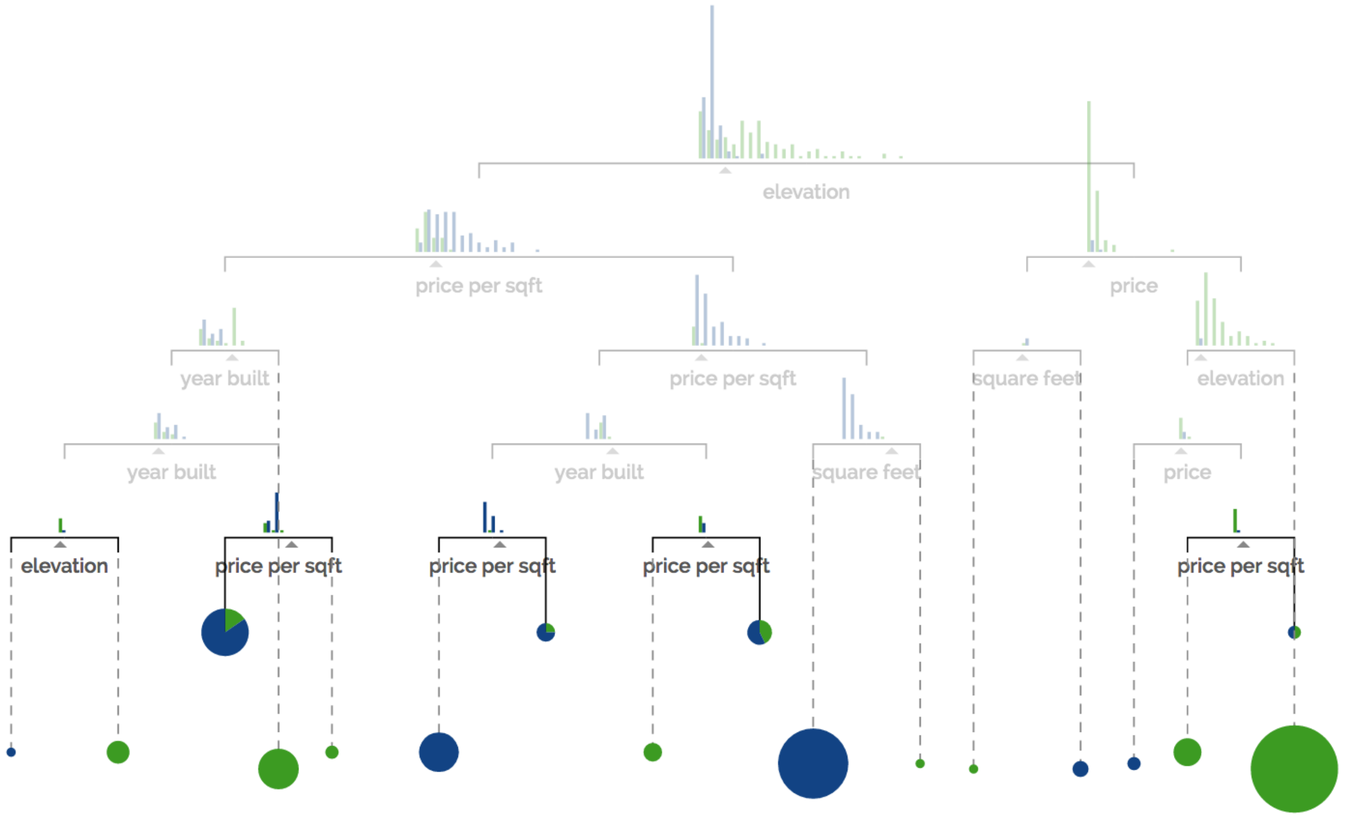


slide credit: Ameet Talwalkar

Decision Tree — Training

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

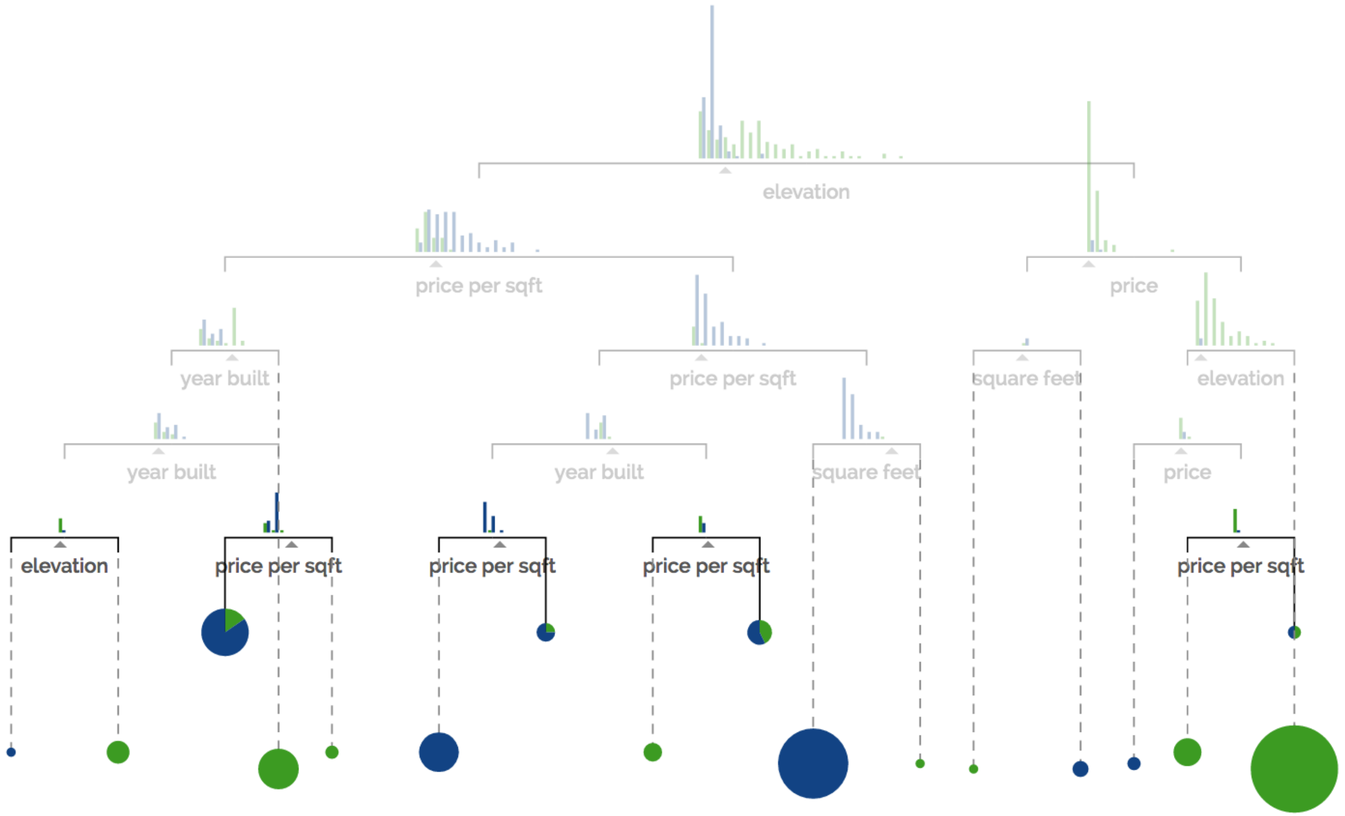


slide credit: Ameet Talwalkar

Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

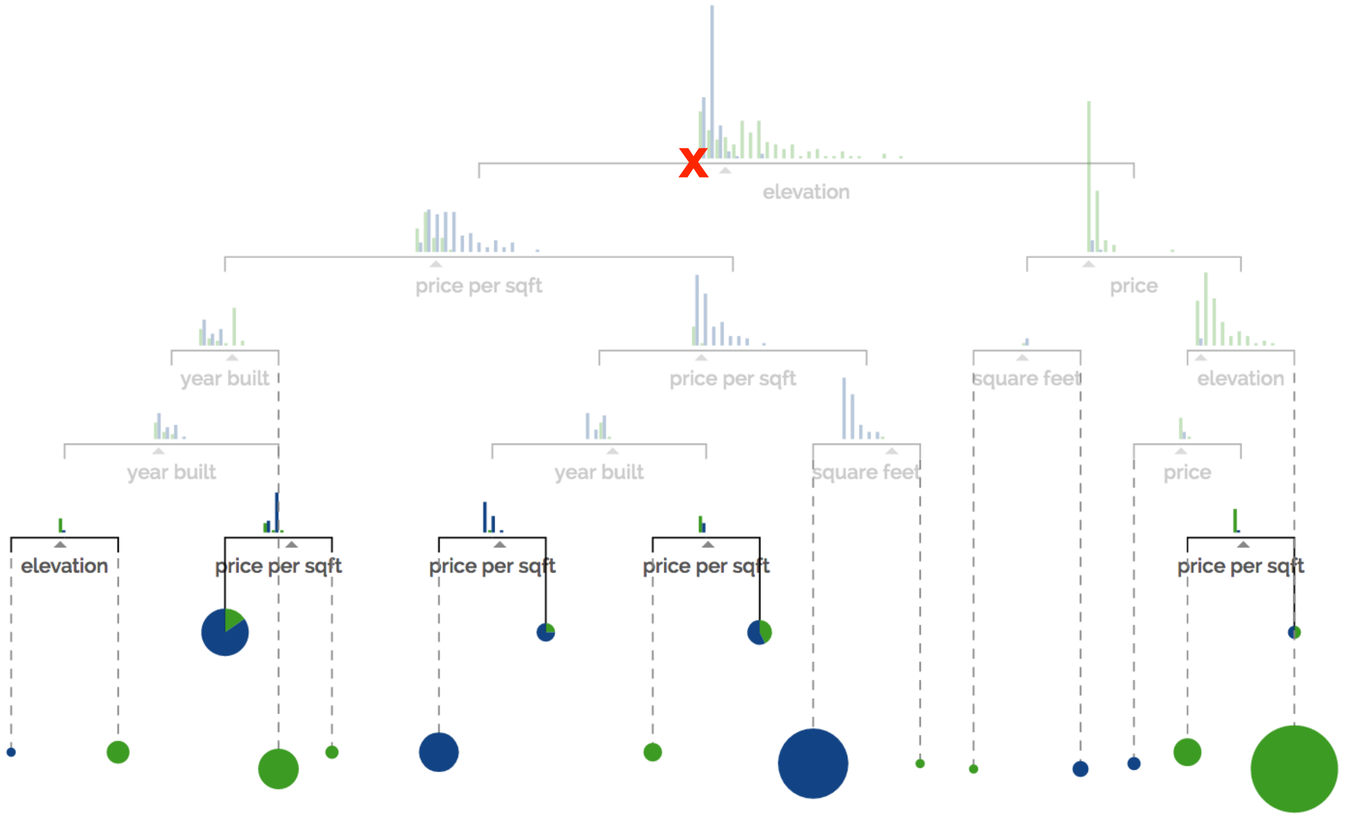


slide credit: Ameet Talwalkar

Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

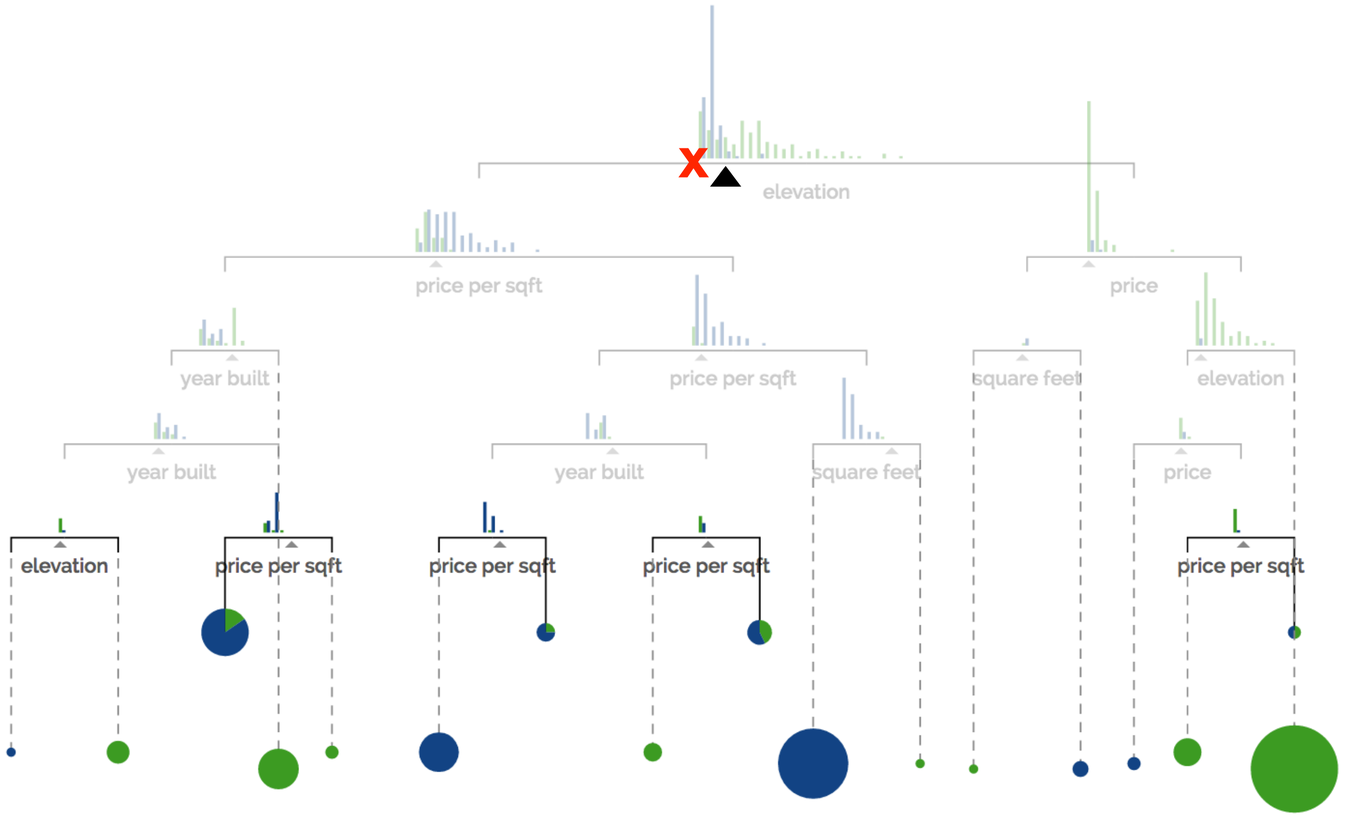


slide credit: Ameet Talwalkar

Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

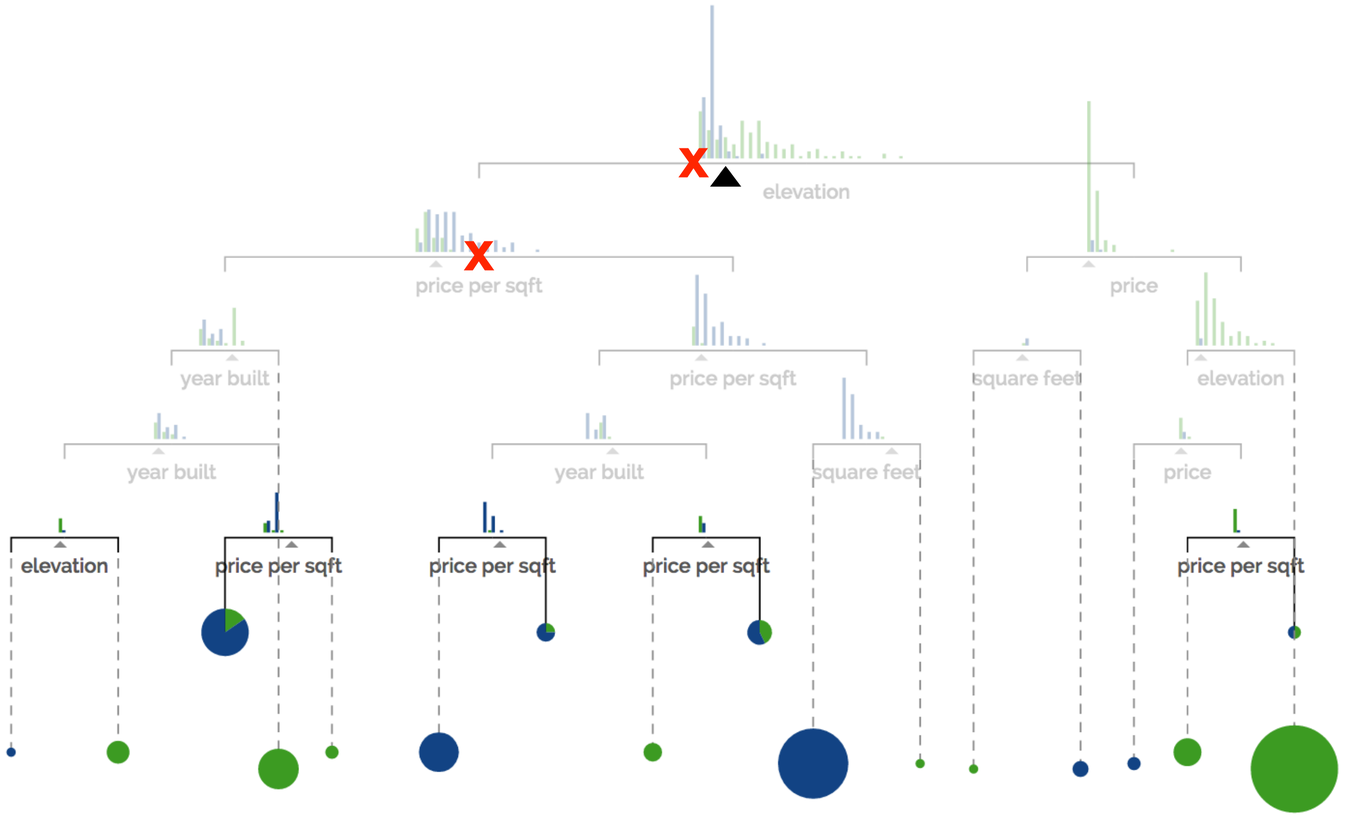


slide credit: Ameet Talwalkar

Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

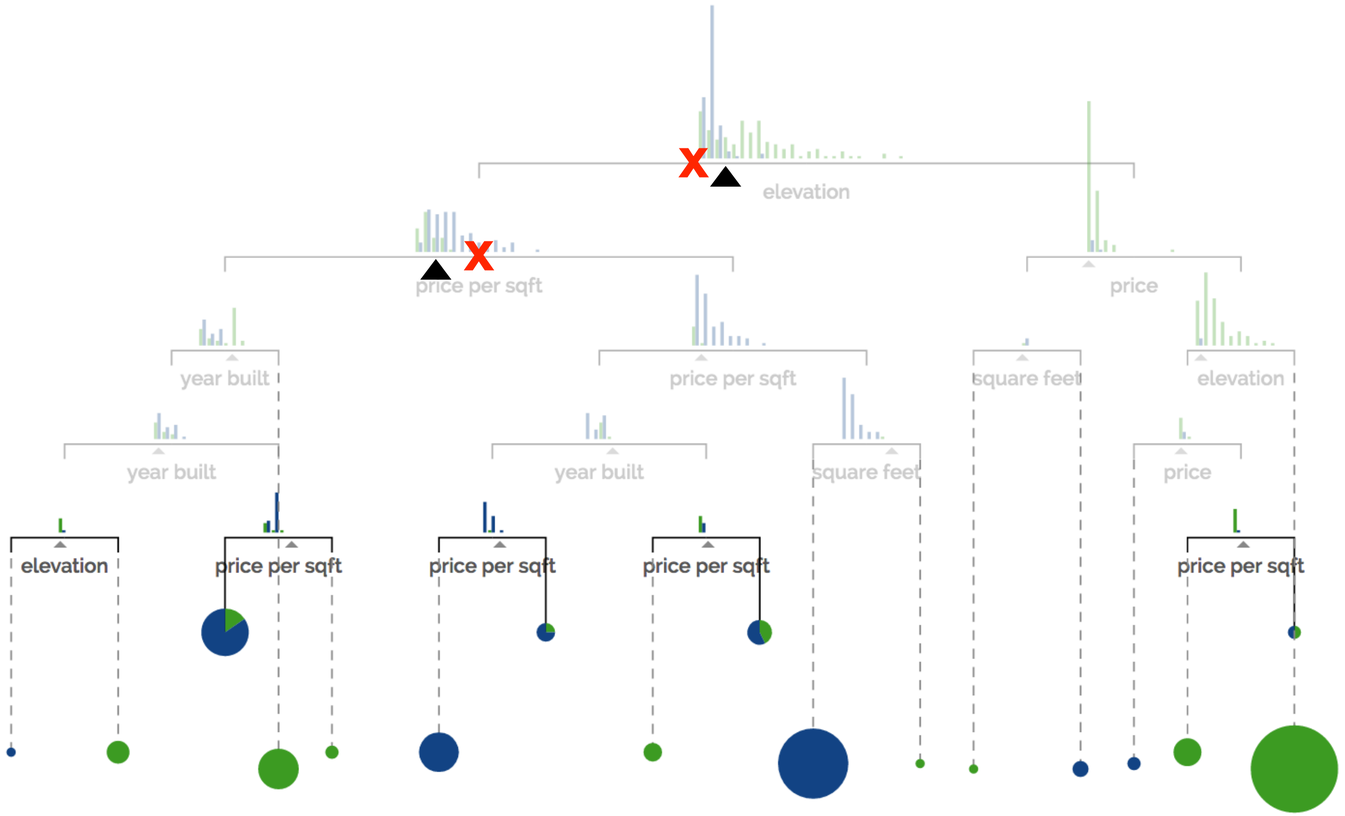


slide credit: Ameet Talwalkar

Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

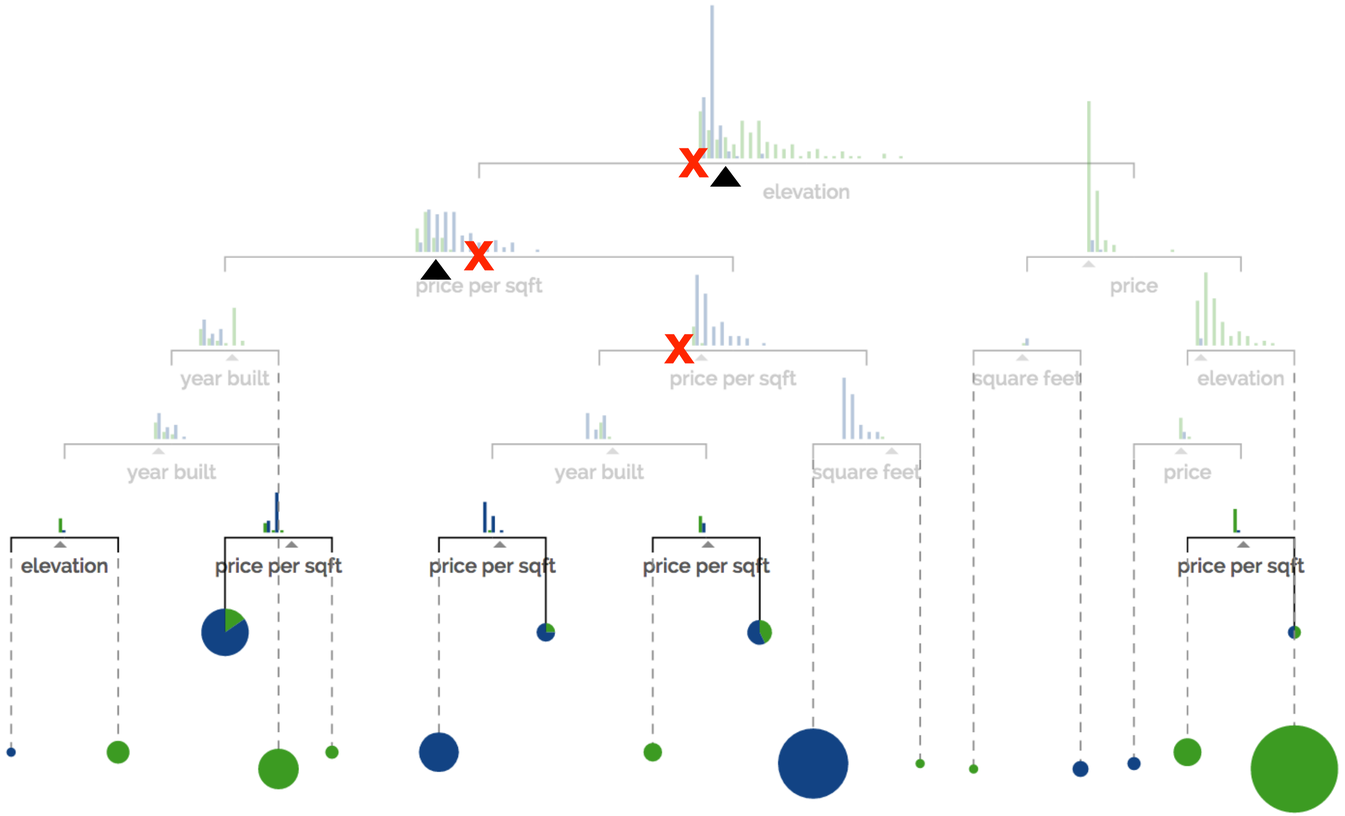


slide credit: Ameet Talwalkar

Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

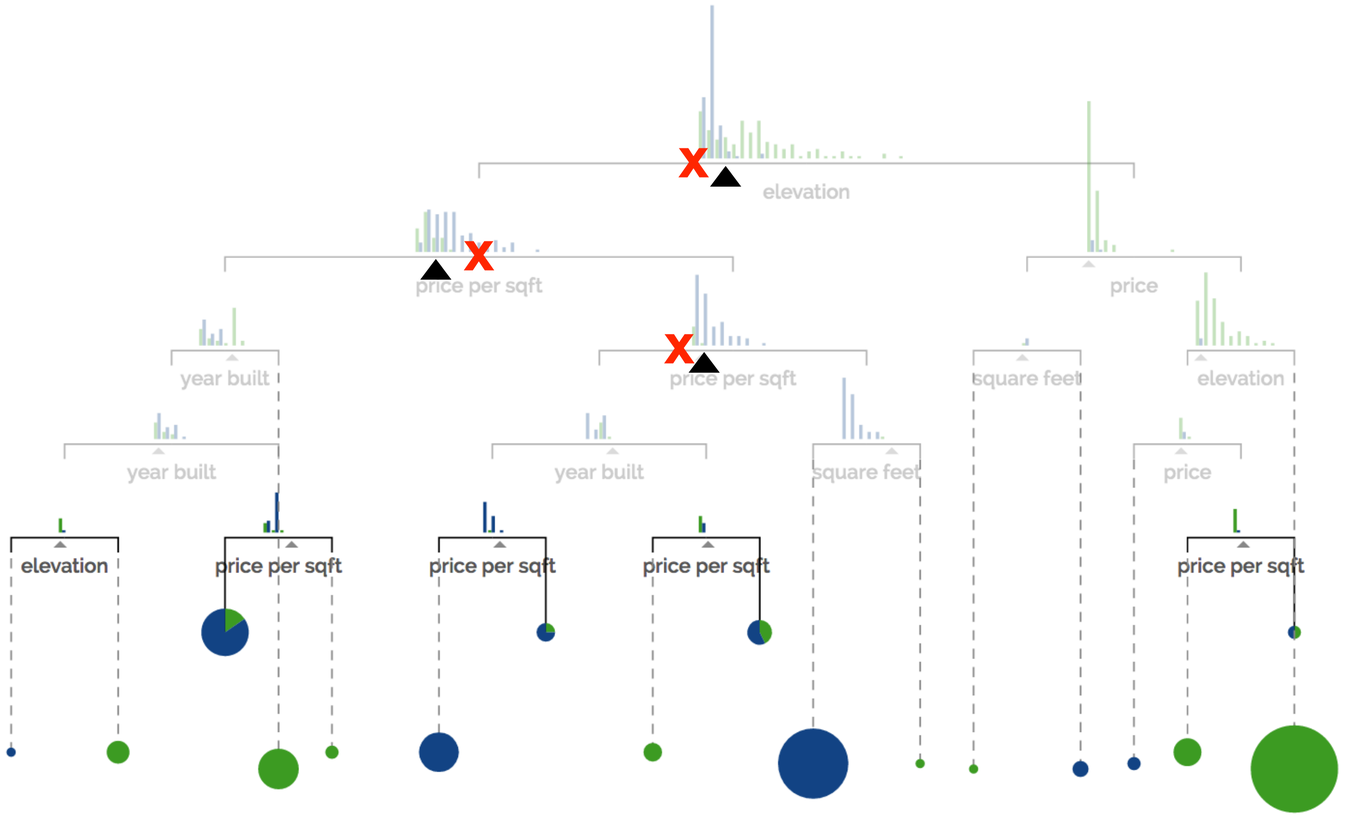


slide credit: Ameet Talwalkar

Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

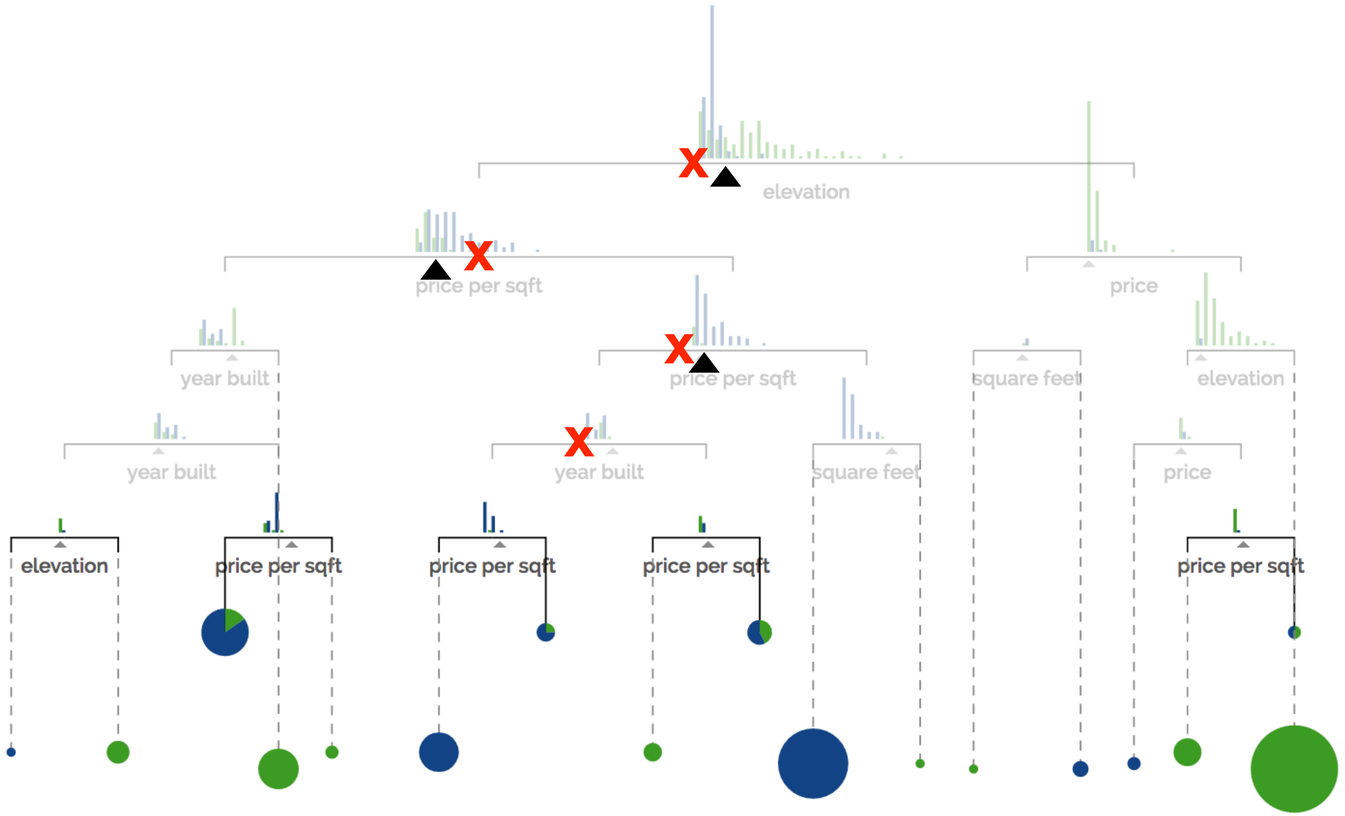


slide credit: Ameet Talwalkar

Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

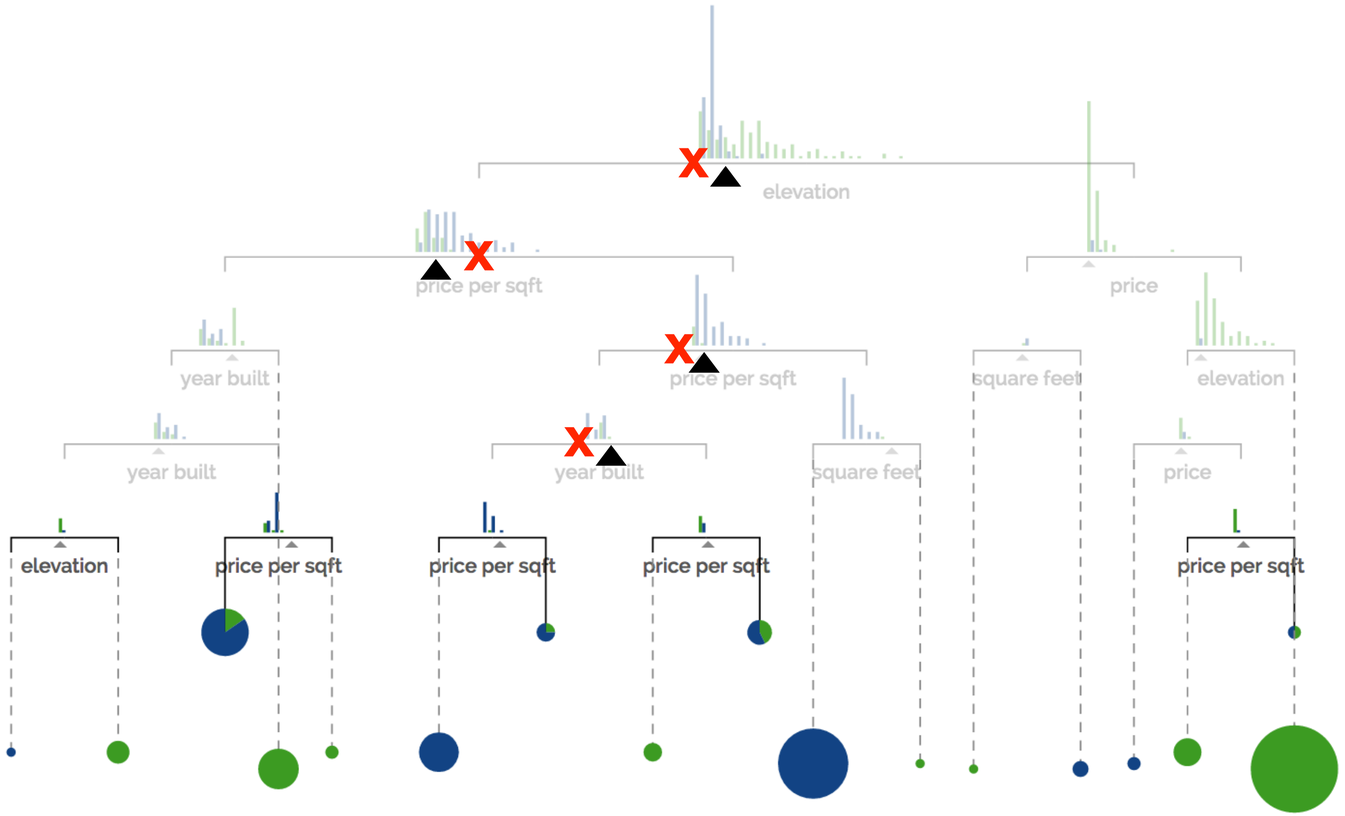


slide credit: Ameet Talwalkar

Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

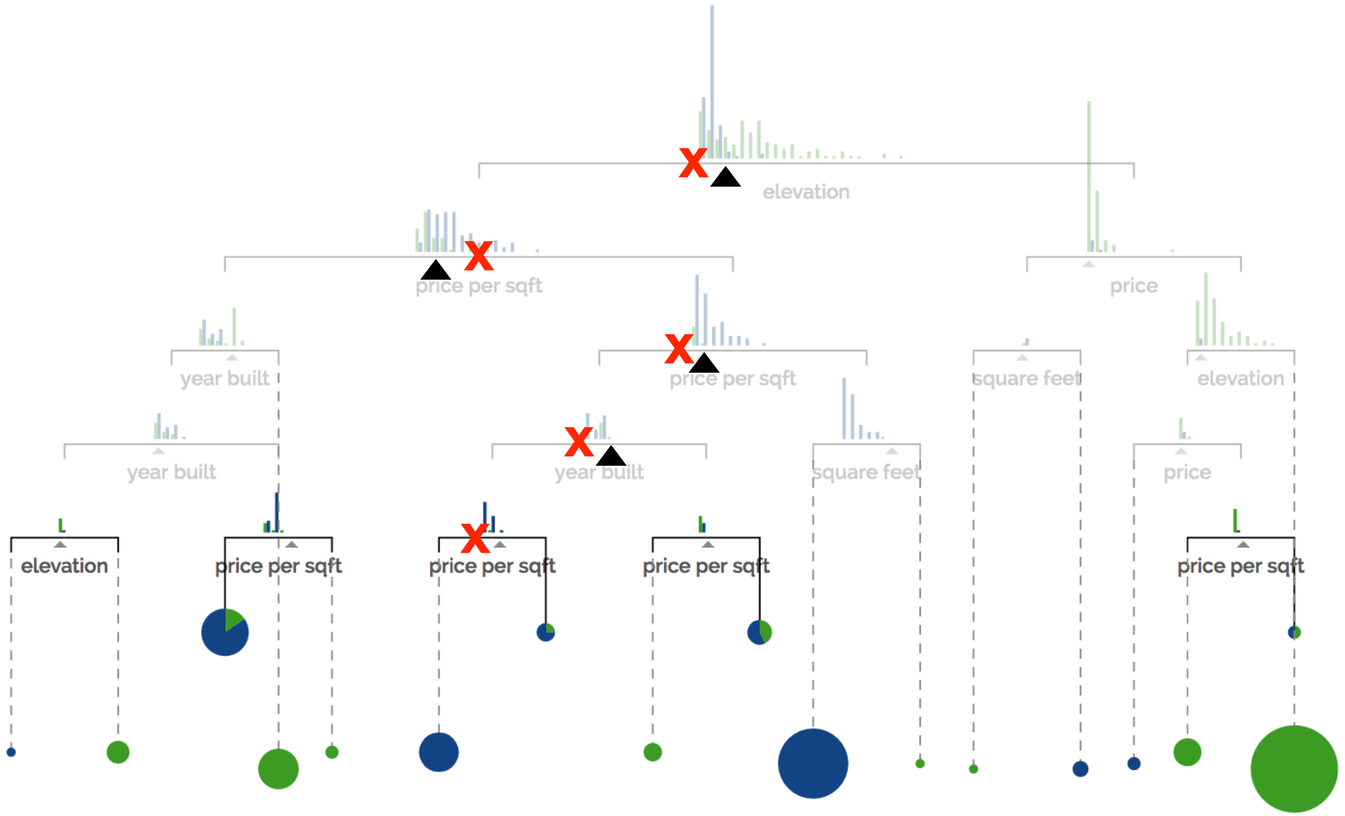


slide credit: Ameet Talwalkar

Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

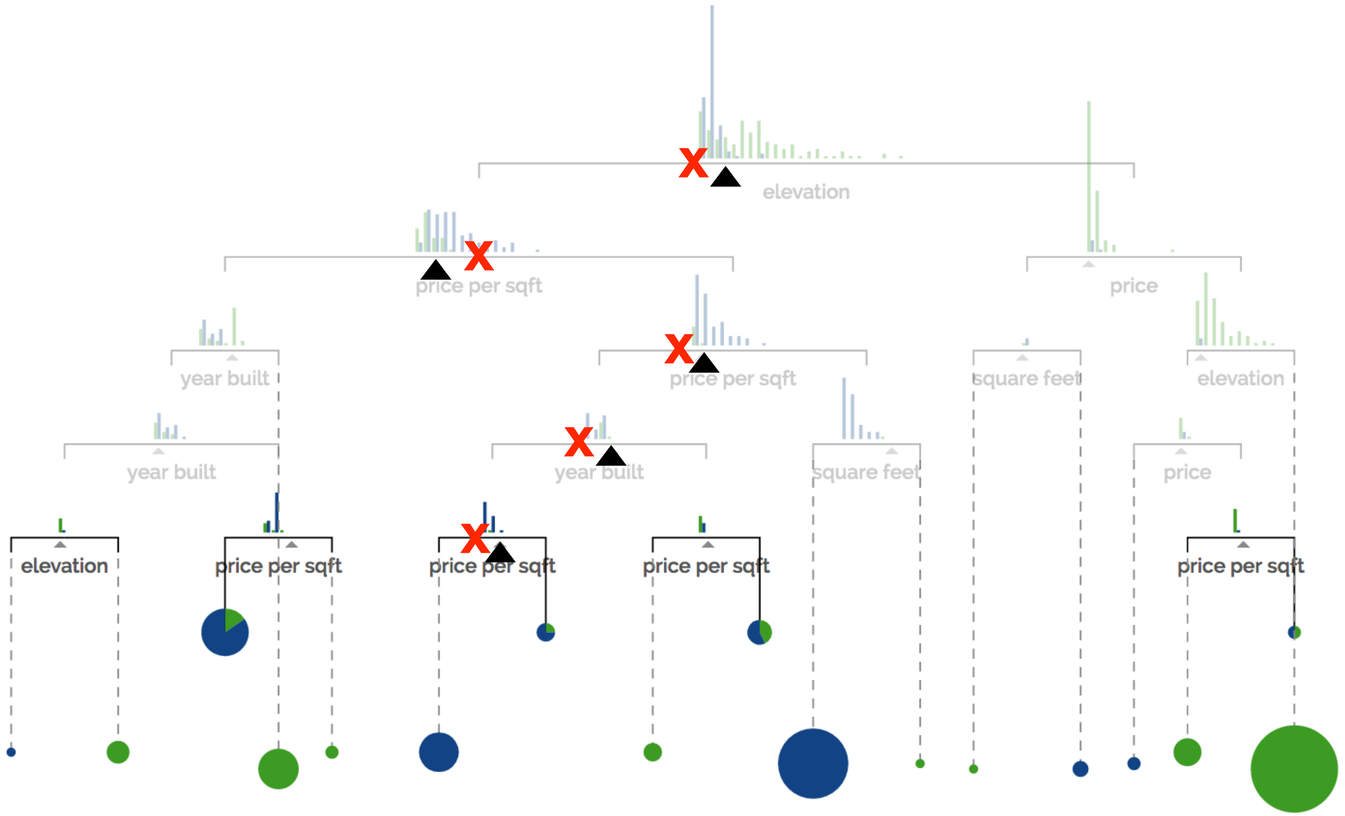


slide credit: Ameet Talwalkar

Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

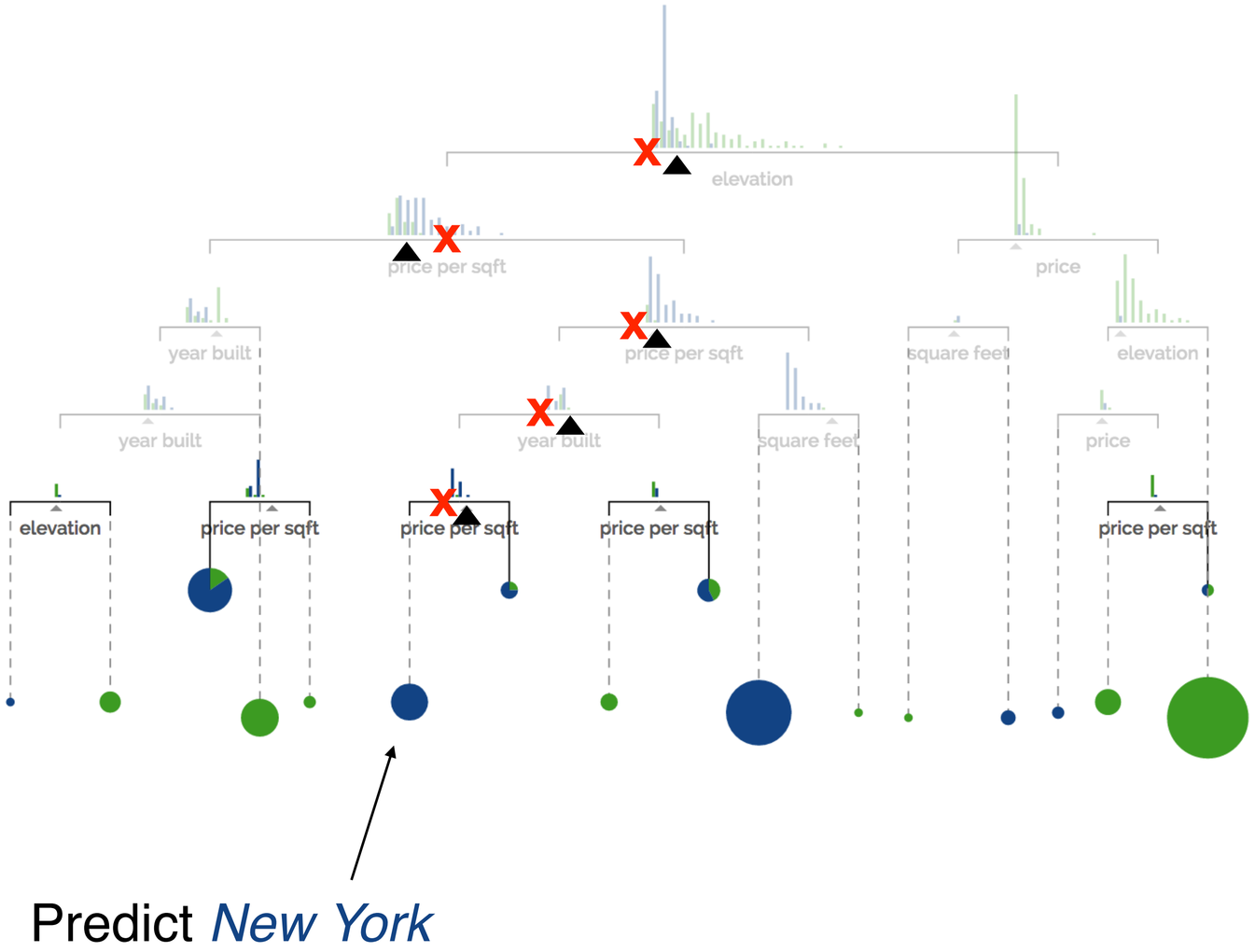


slide credit: Ameet Talwalkar

Decision Tree — Prediction

Suppose you're trying to predict whether a house is located in *San Francisco* or *New York* based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



Predict *New York*

slide credit: Ameet Talwalkar

Decision tree learning pseudocode

- function learn(dataset X, labels Y):
  - if stopping_criterion:
    - n ← new leaf, n.prediction = majority(Y)
  - else:
    - splits ← list of allowed split tests # e.g., feature j = value
    - for s in splits:
      - criterion[s] ← evaluate(X, Y, s) # compute split criterion
    - best_s ← argmax(criterion)
    - mask ← apply best_s to get T/F for each row of X
    - n ← new node
    - n.split = best_s
    - n.left ← learn(X[mask,:], Y[mask])
    - n.right ← learn(X[~mask,:], Y[~mask])
  - return n

Categorical, continuous features

- Often we have features with more than two values
  - color ∈ {red, green, blue, cyan, magenta, yellow}
  - length ∈ [2cm, 7cm]
- Typically we still use ***binary*** splits when learning a tree
  - decision tree learning works better if we grow gradually
- E.g., [color = cyan] or [length ≤ 4.2cm]
- Might need to test several possible splits per feature
  - but still finitely many (even for continuous features!)

Splitting criteria

- Common criteria
  - minimize training error
  - maximize mutual information → ID3
  - minimize Gini impurity criterion → CART

Splitting criteria

- Common criteria
  - minimize training error
  - maximize mutual information → ID3
  - minimize Gini impurity criterion → CART

Mutual information as a splitting criterion

- When we run a decision tree, each example sorts to some leaf l and has true label y
- A good decision tree has high mutual information between l and y (knowing l is helpful in predicting y)
- Splitting criterion: which split increases this mutual information the most, as measured on the training set?

Underfitting and Overfitting in Decision Tree Learning

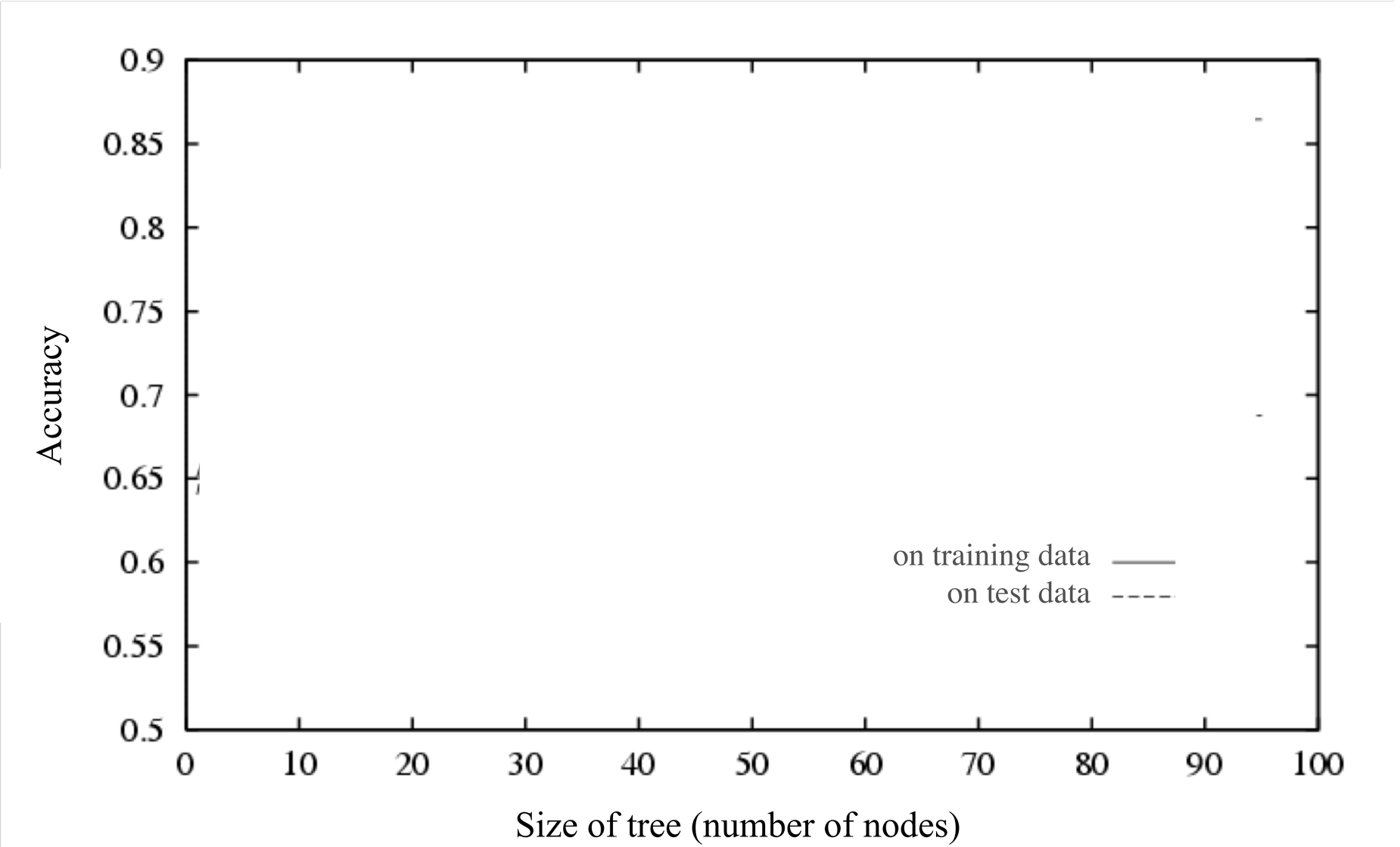


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

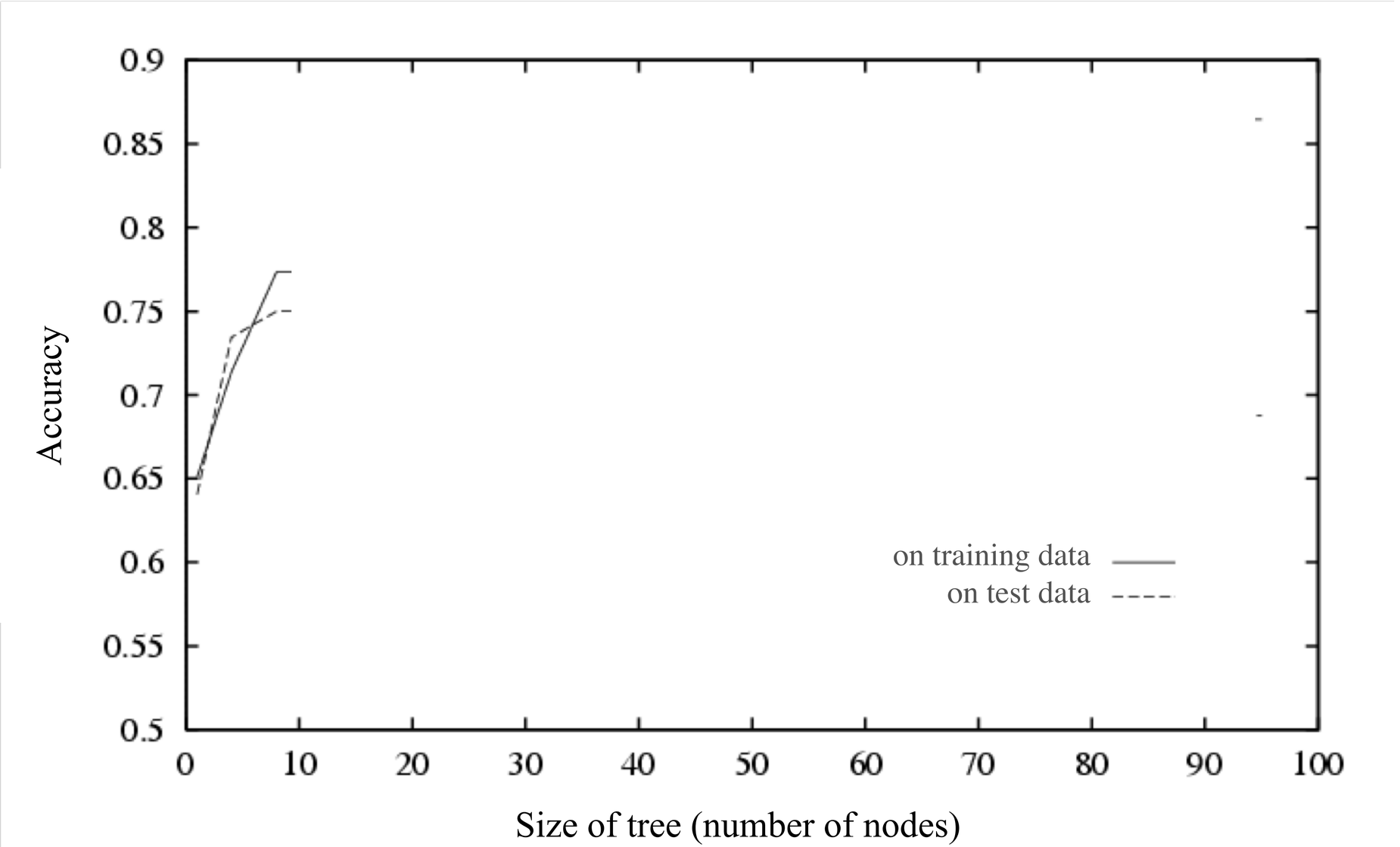


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

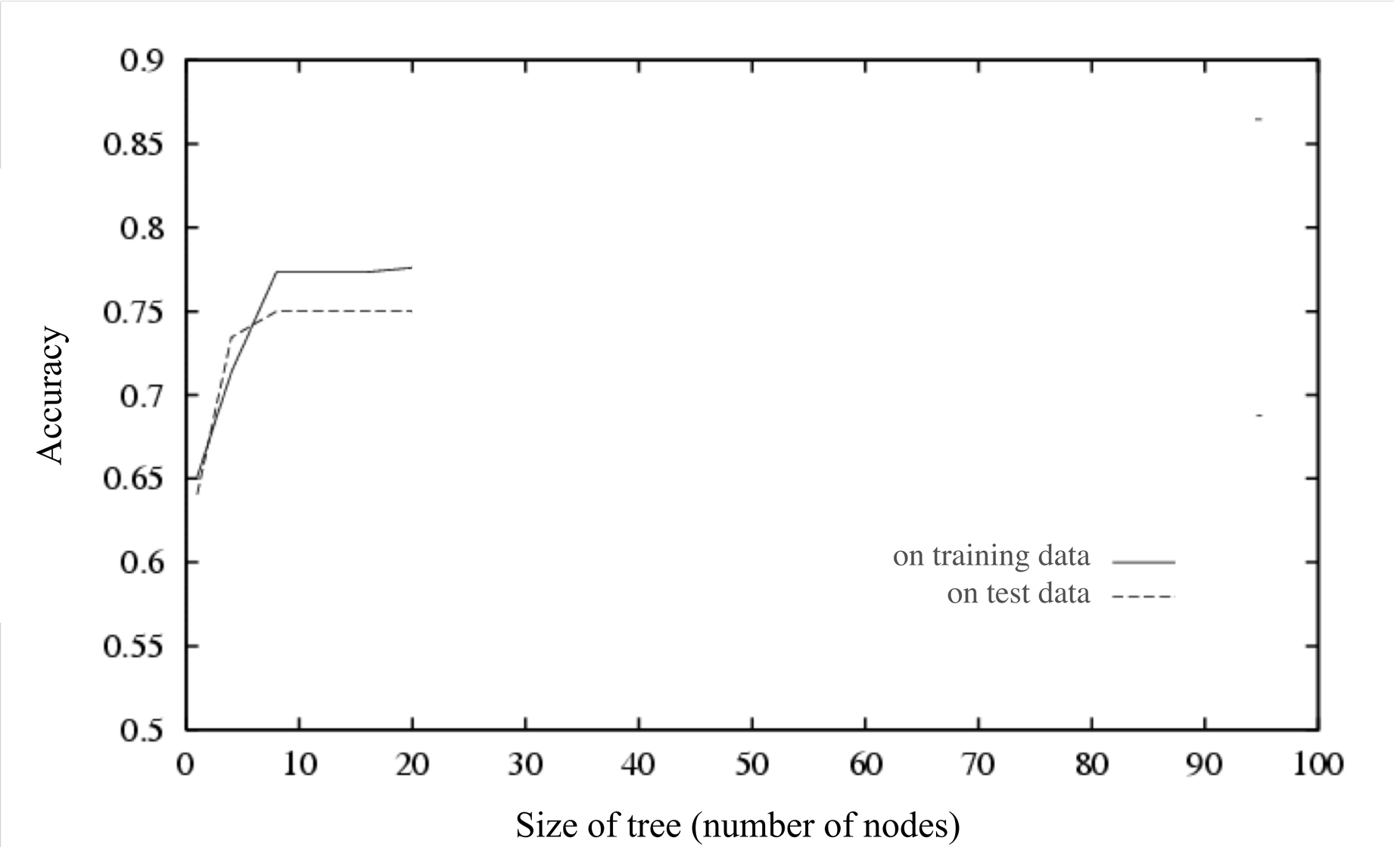


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

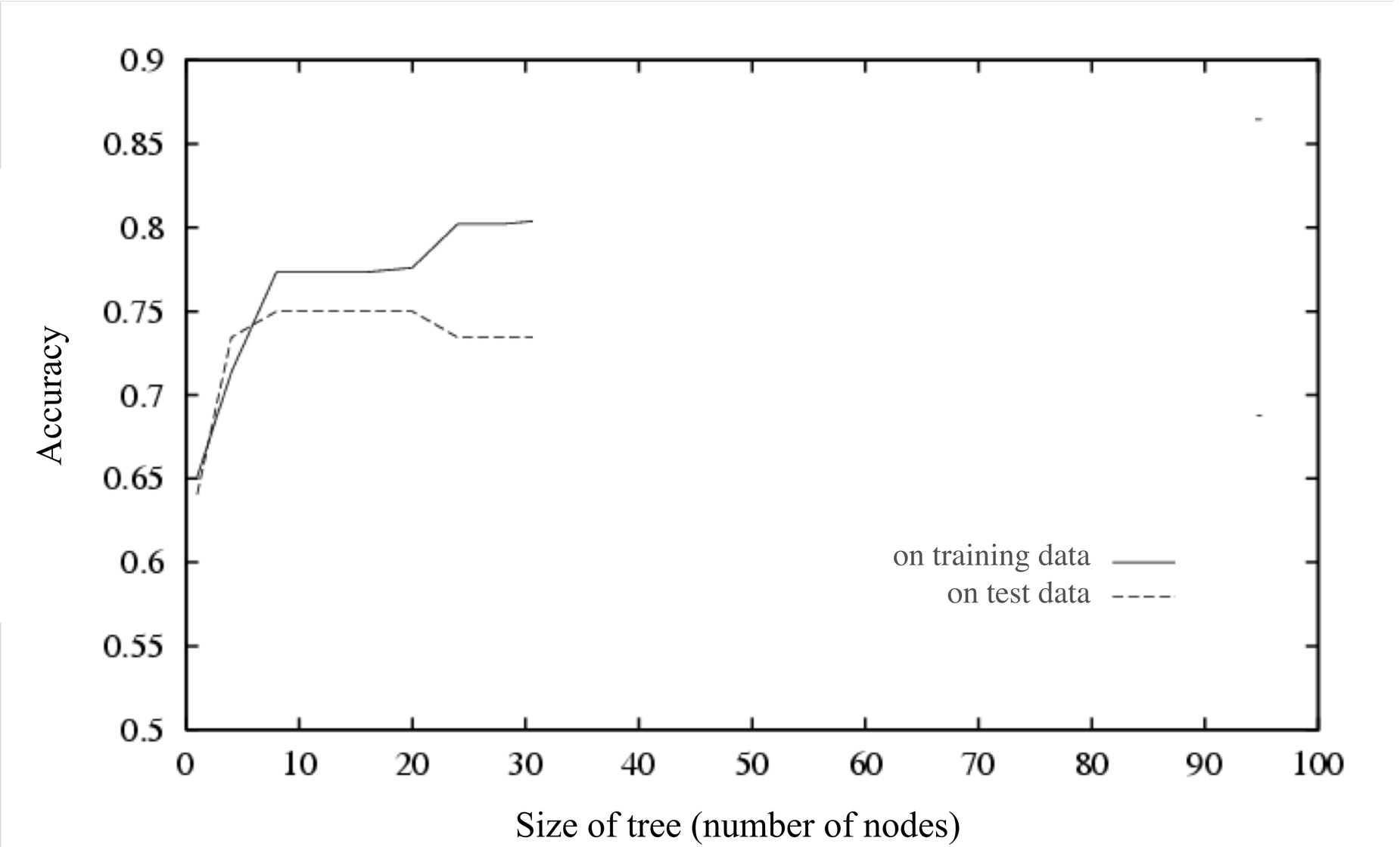


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

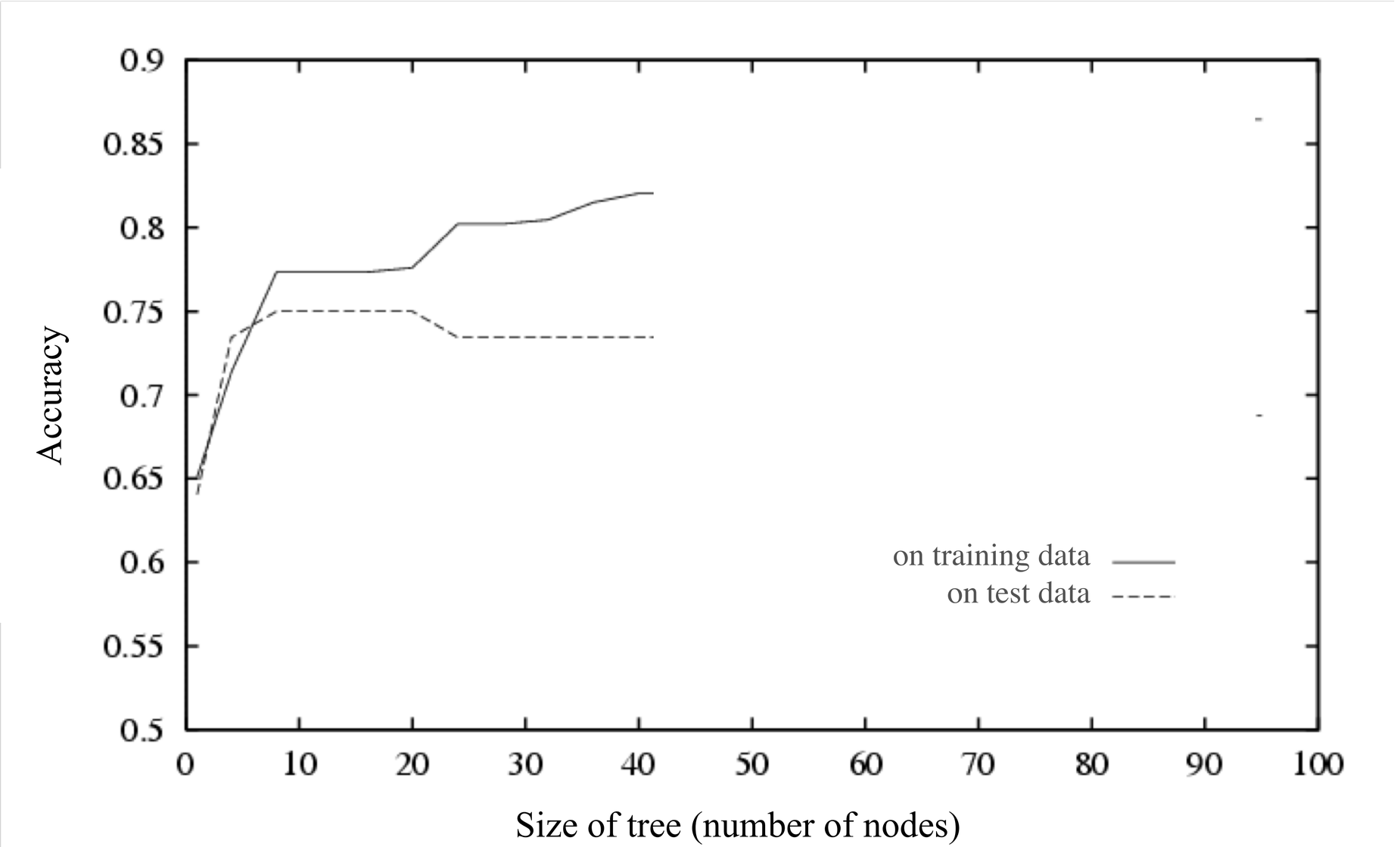


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

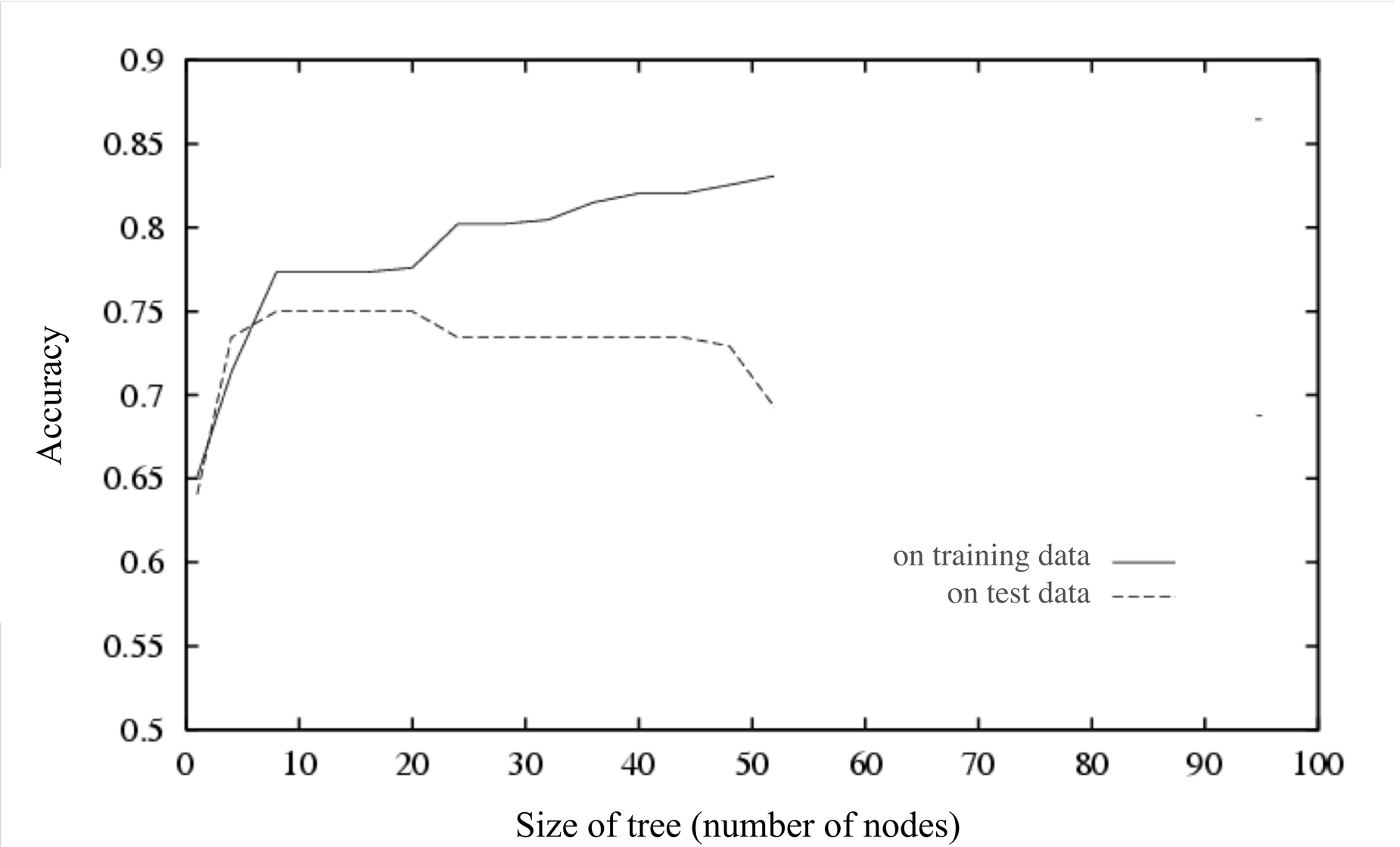


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

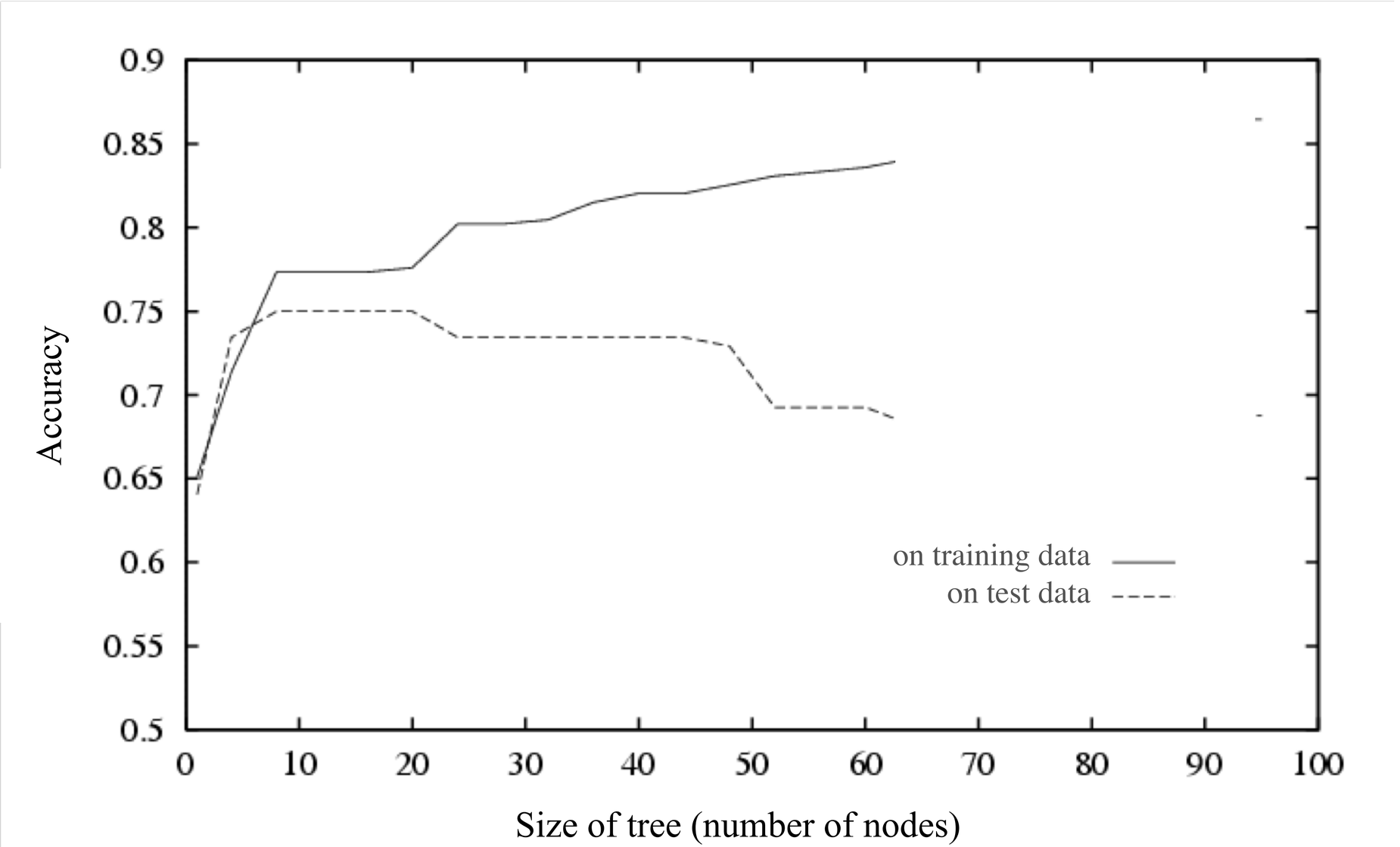


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

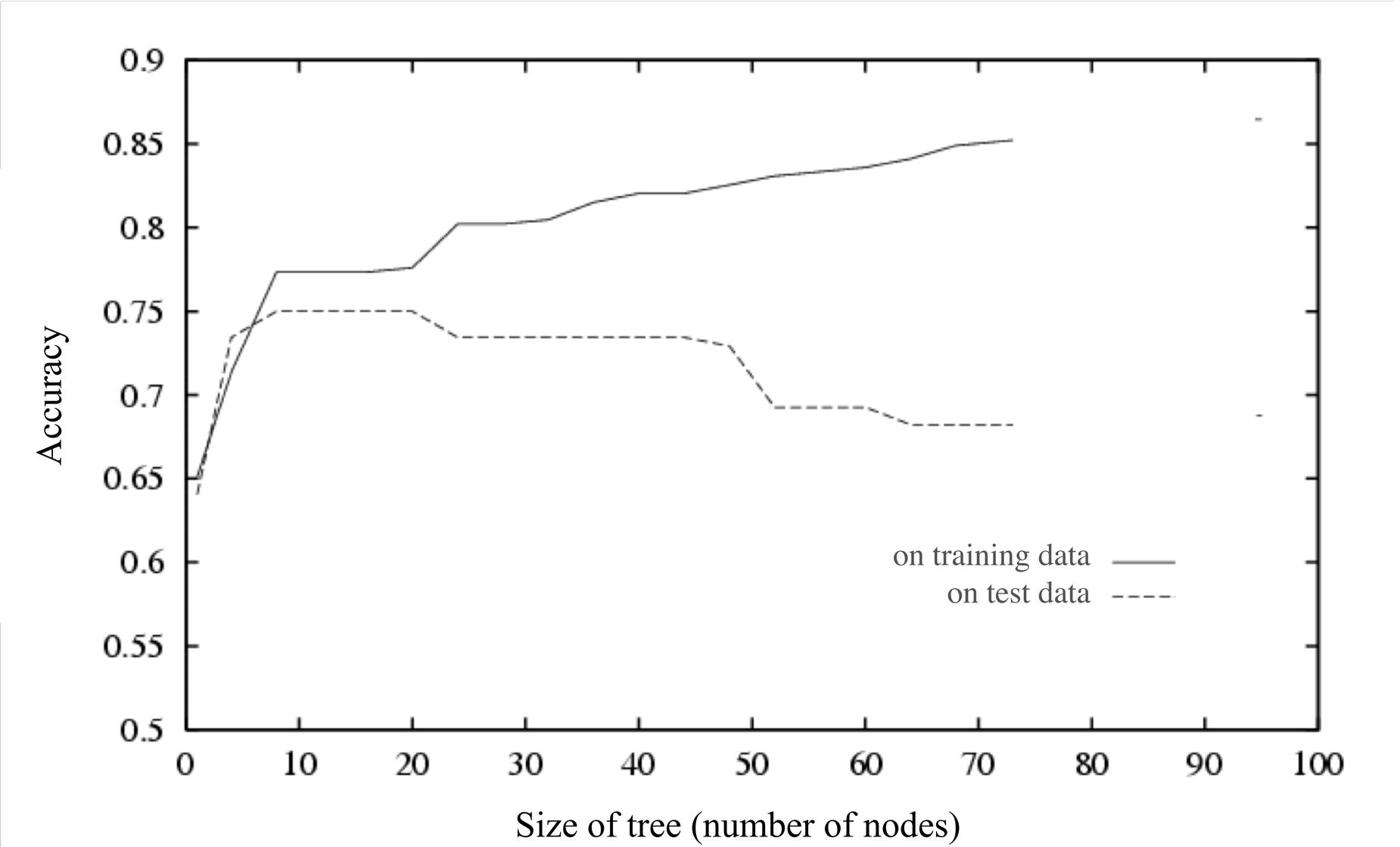


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

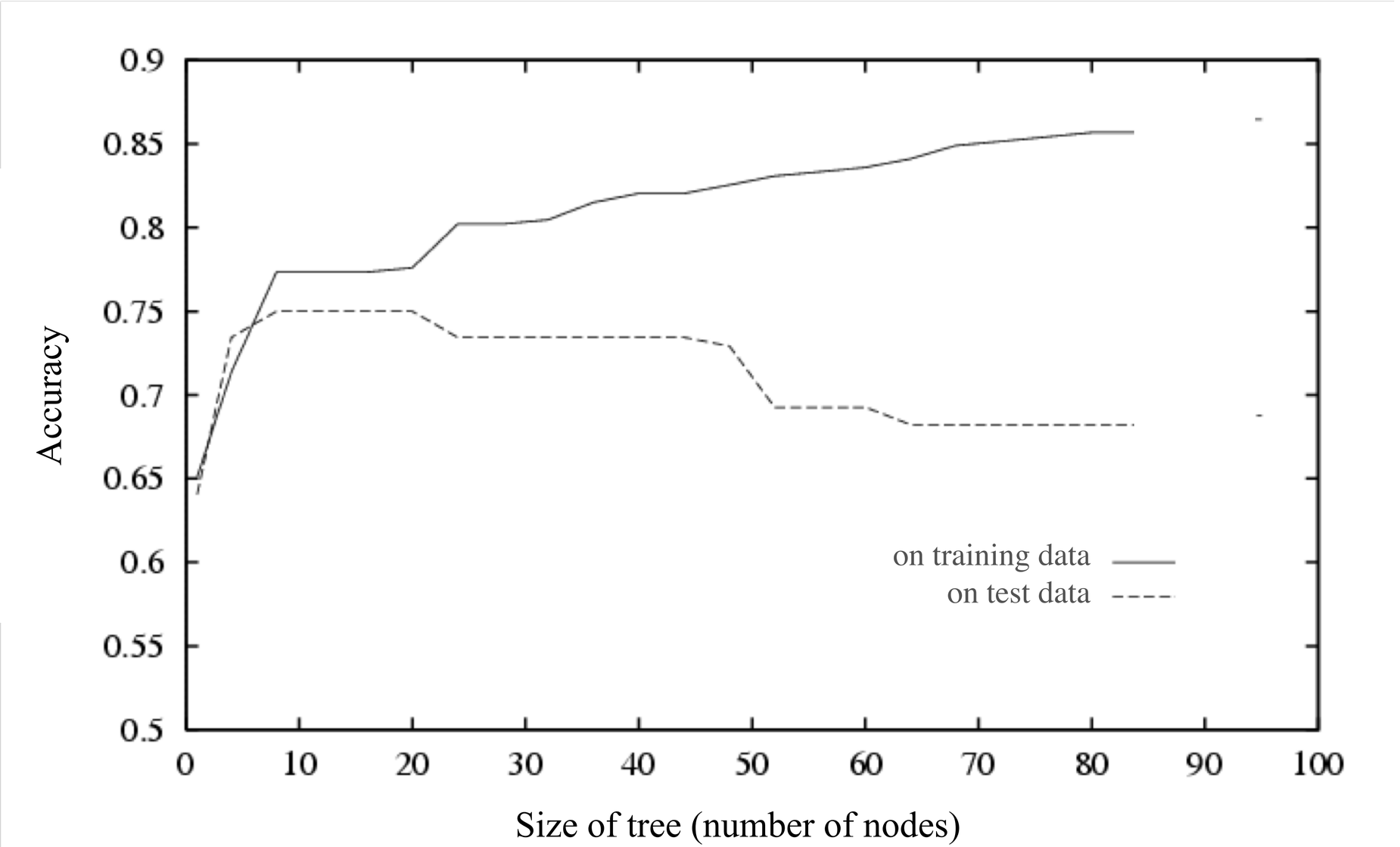


Figure from Tom Mitchell

Underfitting and Overfitting in Decision Tree Learning

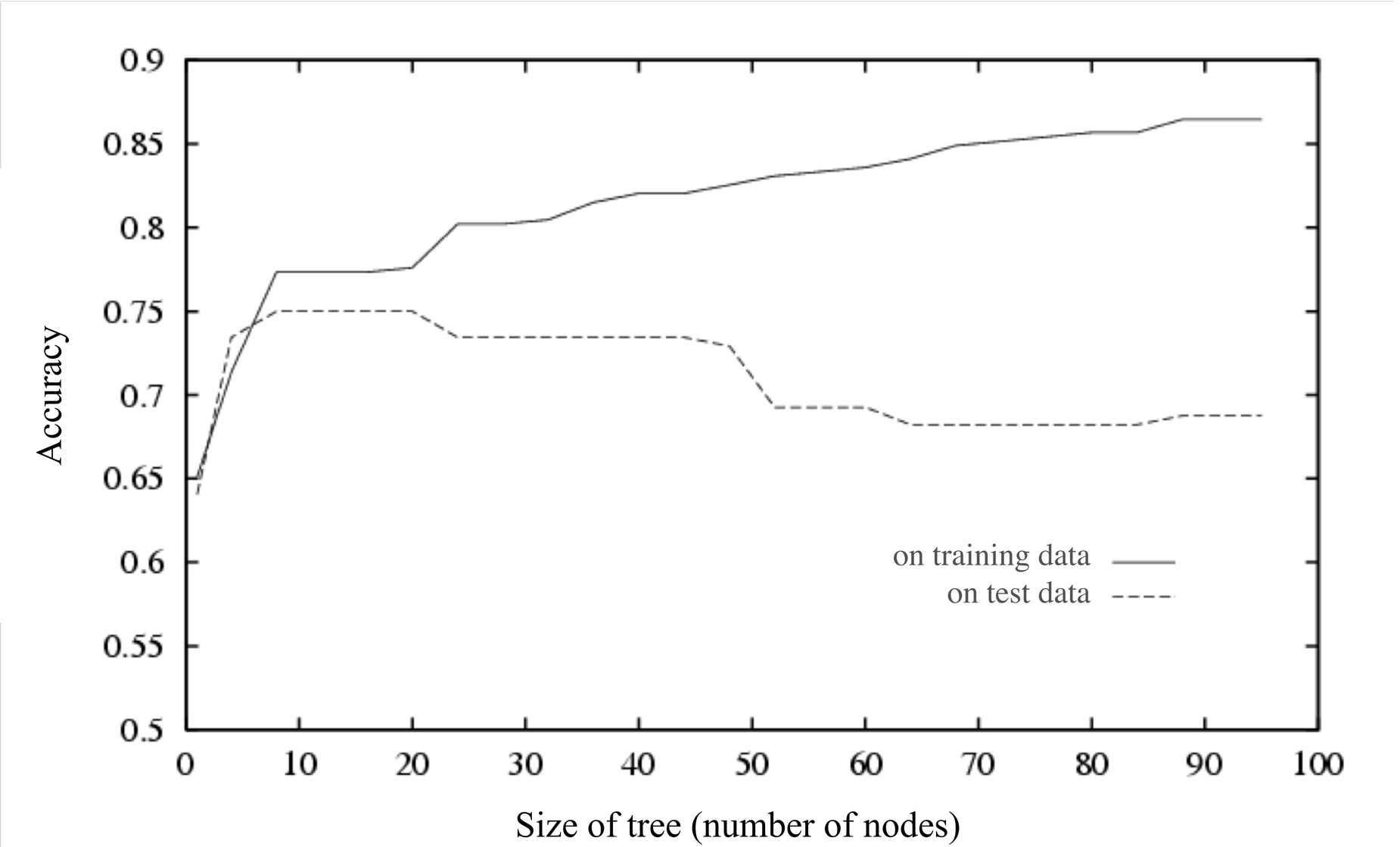


Figure from Tom Mitchell

How can we avoid overfitting in decision trees?

- Need a measure of simplicity/complexity
  - depth, number of nodes/leaves, size (# examples) of leaf
- Some proposals for minimizing these measures:
  - do not grow tree beyond some maximum depth or minimum leaf size
  - do not split if criterion (e.g. increase in MI) is too low
  - stop growing when the change in criterion is not statistically significant
  - grow the entire tree, then prune

Prune

- Prune: undo some splits, making the tree simpler (reduce depth and number of leaves, increase examples per leaf)
- Idea: measure the predicted test error, prune a node if doing so improves prediction
- How do we measure predicted test error?
  - ***hide*** some data from the learner — take what would have been our training data and split it
  - use one part as the actual training set (learner sees this)
  - use other part to stand in for test set (learner doesn't see this) — called ***validation set***
- Original test set stays the same — can still use it to evaluate final pruned tree
  - why can't we use test set to prune?

Train/ validation/ test split

validation is sometimes called ***hold-out*** set

- Start with available data $\mathscr{D}$
- Split into three parts:
  - $\mathscr{D}_{\text{train}}$: learning the tree
  - $\mathscr{D}_{\text{val}}$: pruning
  - $\mathscr{D}_{\text{test}}$: final evaluation
- Tension:
  - bigger $\mathscr{D}_{\text{train}}$ for better initial learning
  - bigger $\mathscr{D}_{\text{val}}$ for better pruning
  - bigger $\mathscr{D}_{\text{test}}$ for accurate evaluation

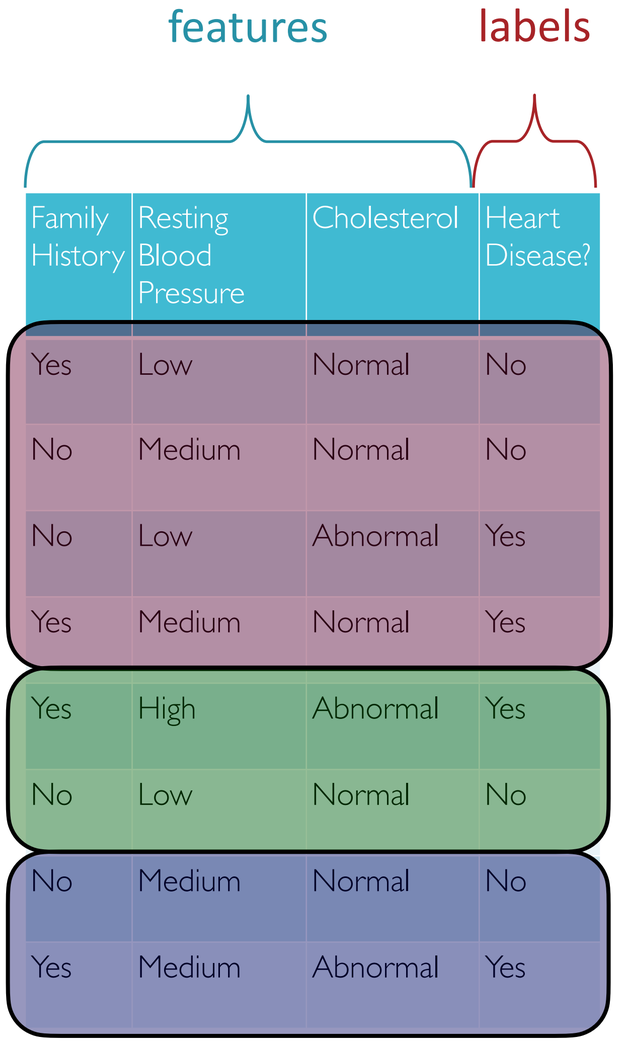
features labels

Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes
No	Low	Normal	No
No	Medium	Normal	No
Yes	Medium	Abnormal	Yes

Train/ validation/ test split

validation is sometimes called ***hold-out*** set

- Start with available data $\mathscr{D}$
- Split into three parts:
  - $\mathscr{D}_{\text{train}}$: learning the tree
  - $\mathscr{D}_{\text{val}}$: pruning
  - $\mathscr{D}_{\text{test}}$: final evaluation
- Tension:
  - bigger $\mathscr{D}_{\text{train}}$ for better initial learning
  - bigger $\mathscr{D}_{\text{val}}$ for better pruning
  - bigger $\mathscr{D}_{\text{test}}$ for accurate evaluation



Hidden dependence

- Common gotcha: examples not completely independent
  - surveys given by Bob vs. surveys given by Alice
  - measurements taken from nearby plots of land
  - slow changes over time
- Causes two problems:
  - information leakage: training set contains spoilers about test set (so performance estimates become optimistic)
  - generalization failure: ML depends on similarity between training and test (***iid*** assumption); if something changes, what we learned might not be as relevant

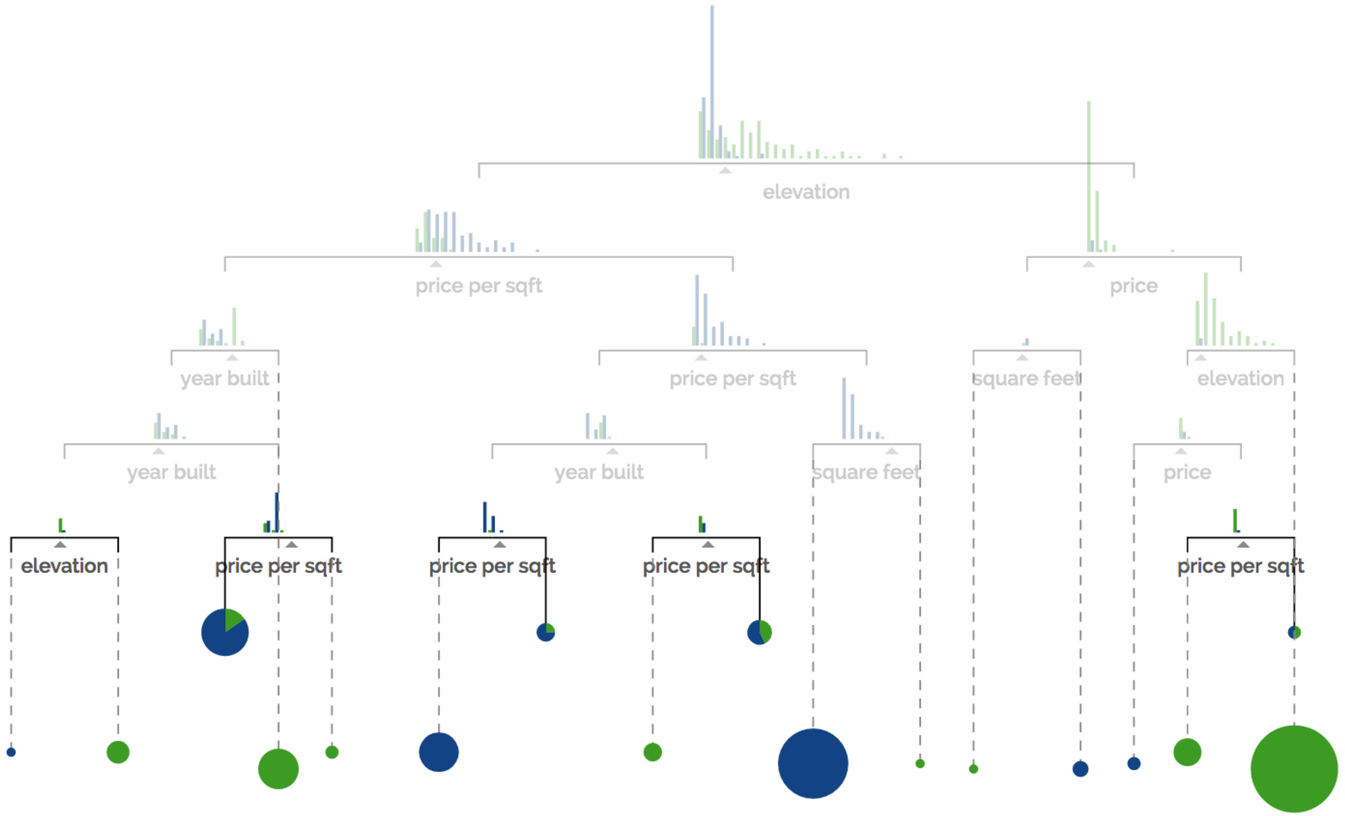
Possible fixes

- Block randomization:
  - sometimes we have information about dependence (who gave the survey, which plot of land, timestamp)
  - randomize ***blocks*** of related examples together to train, validation, or test
    - e.g., a block of time all goes to test, or all of Alice's surveys go to validation — reduces leakage
- Shuffle before partitioning
  - can increase leakage, but better match to iid

Decision Tree — Pruning

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

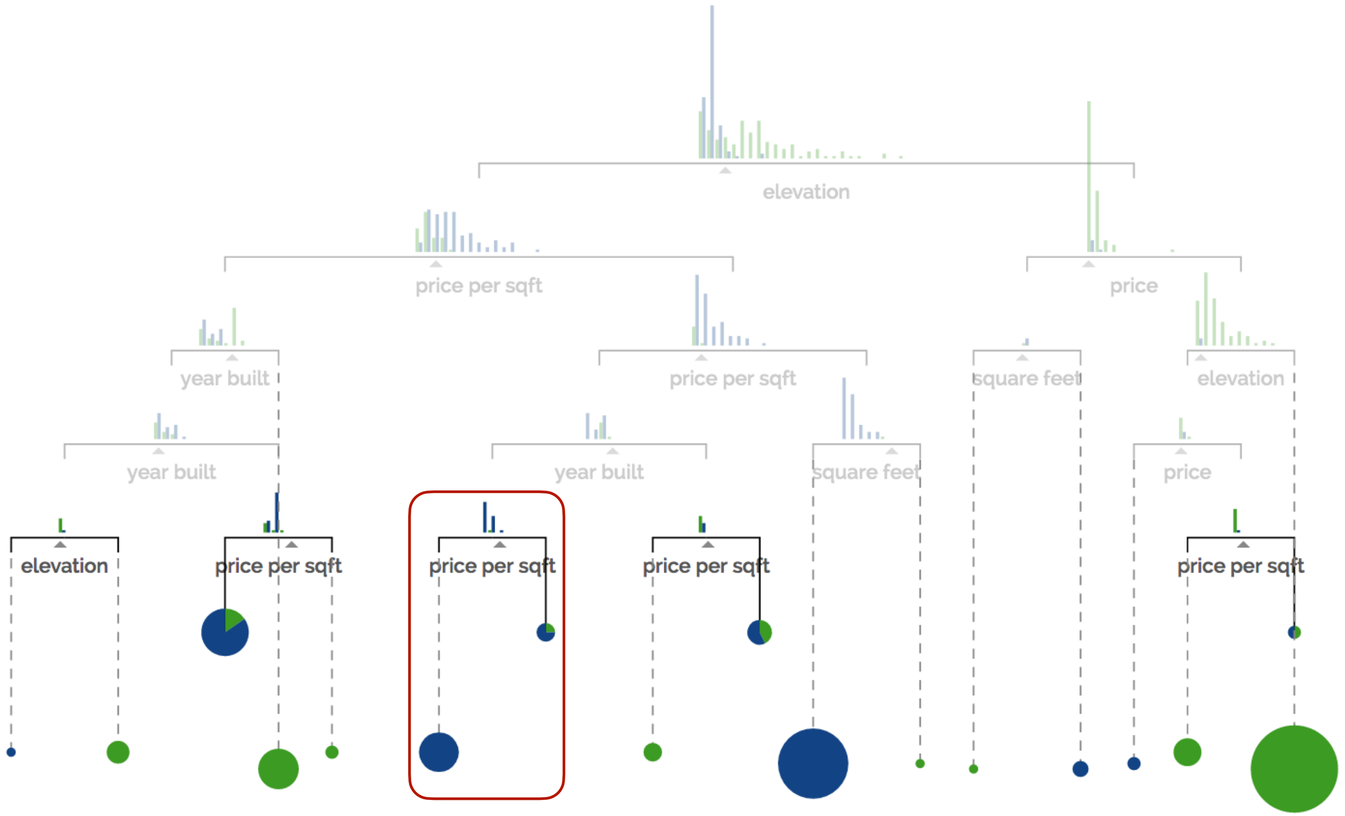


slide credit: Ameet Talwalkar

Decision Tree — Pruning

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms

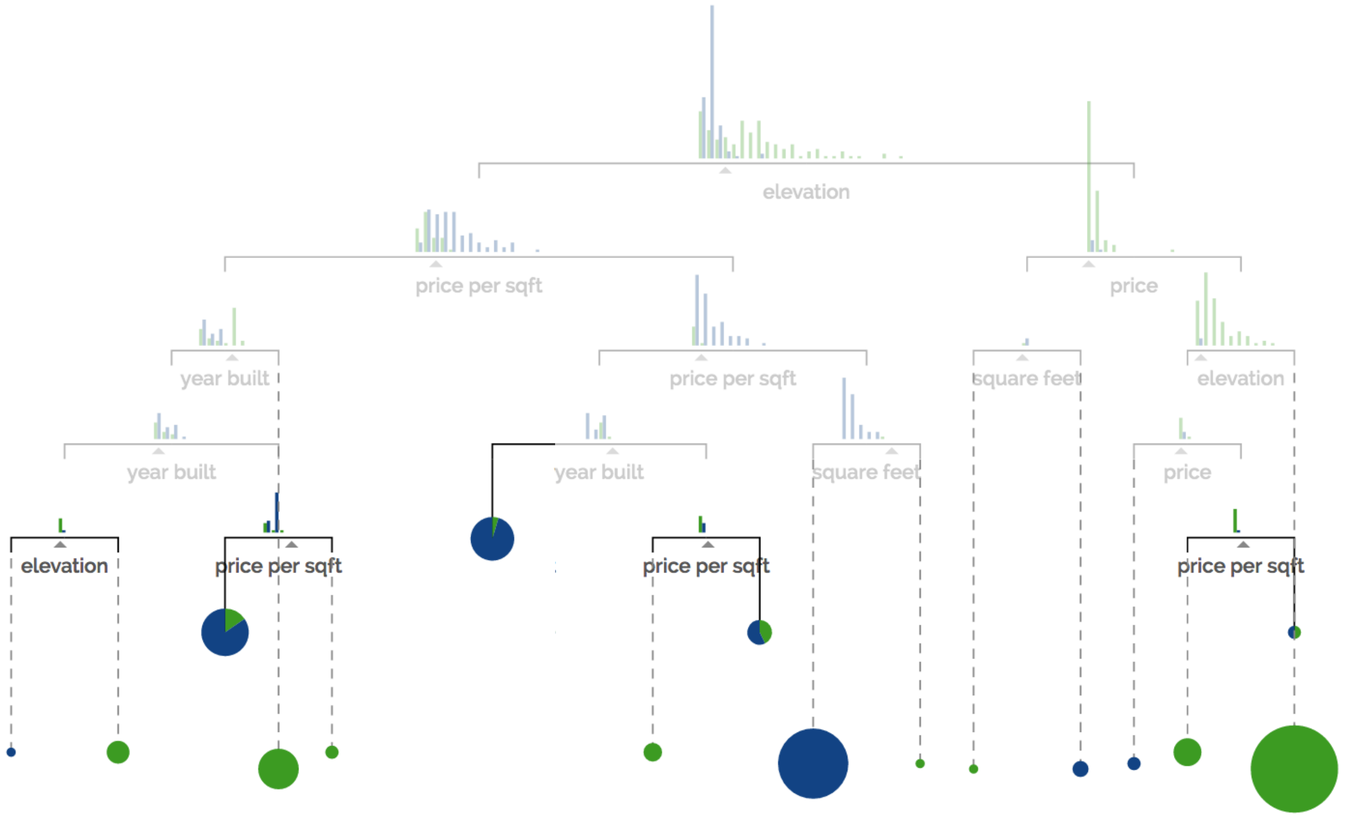


slide credit: Ameet Talwalkar

Decision Tree — Pruning

Suppose you're trying to predict whether a house is located in ***San Francisco*** or ***New York*** based on the following features:

- price
- sq ft
- price per sq ft
- year built
- elevation
- # bathrooms
- # bedrooms



slide credit: Ameet Talwalkar

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

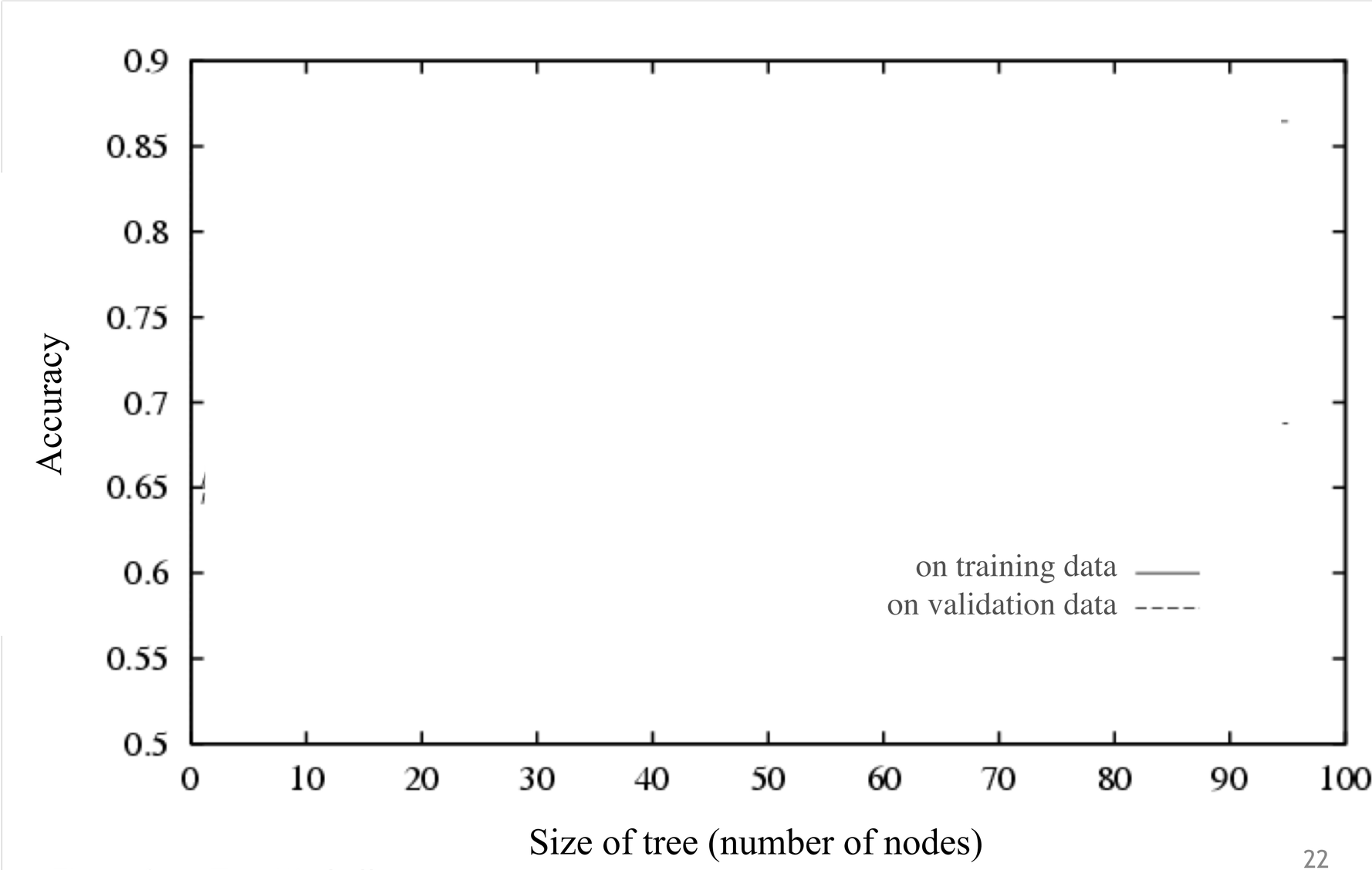


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

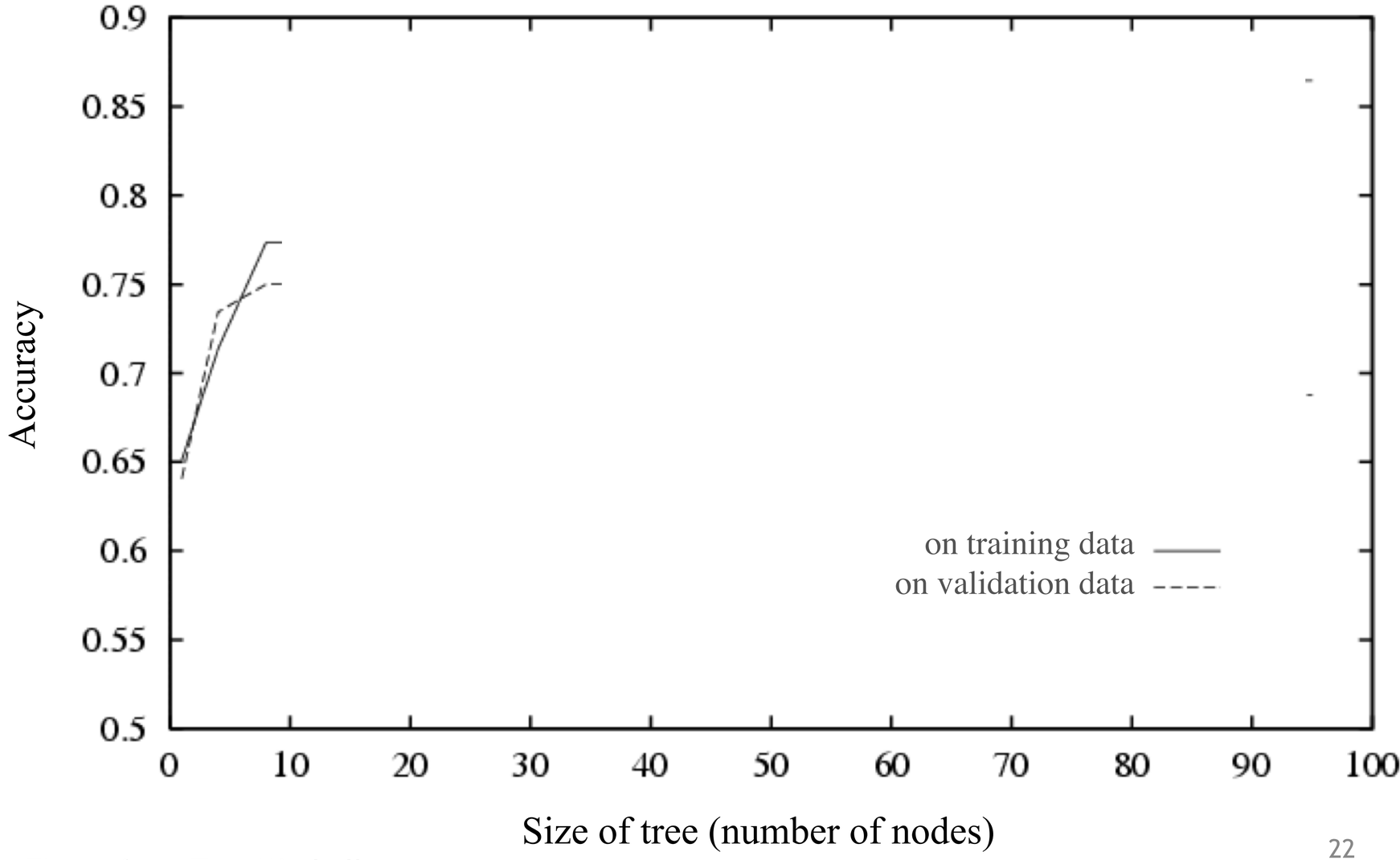


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

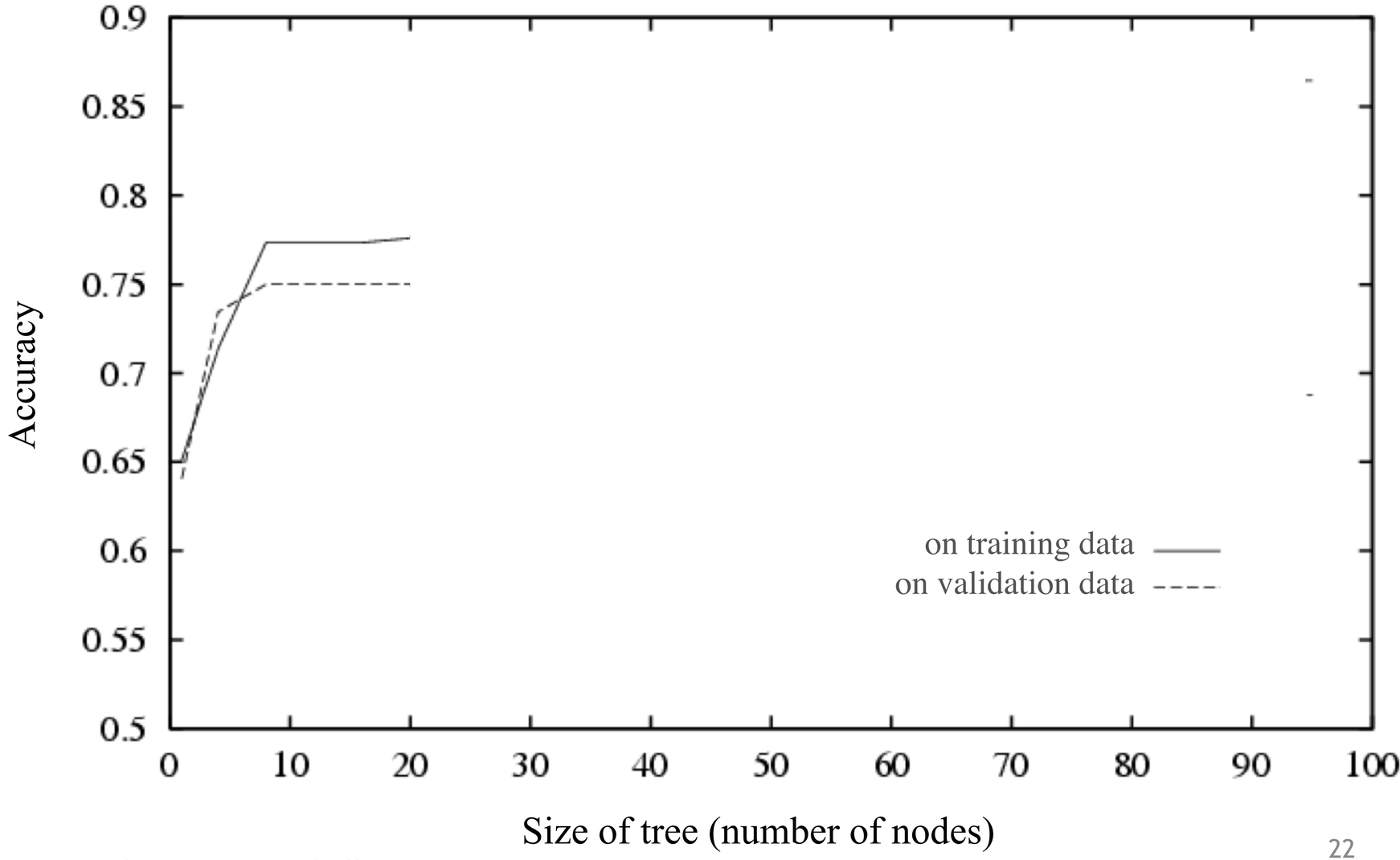


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

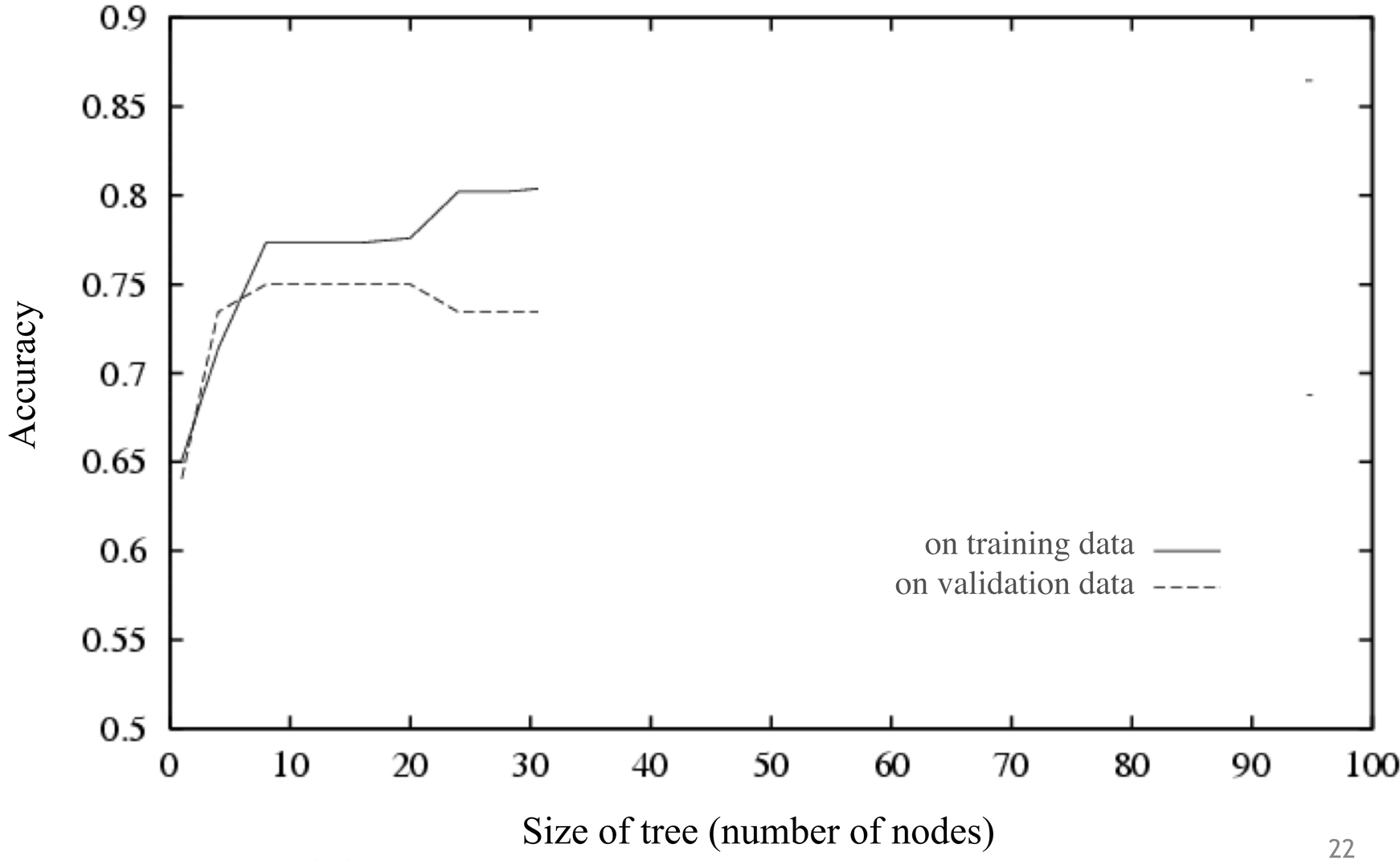


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

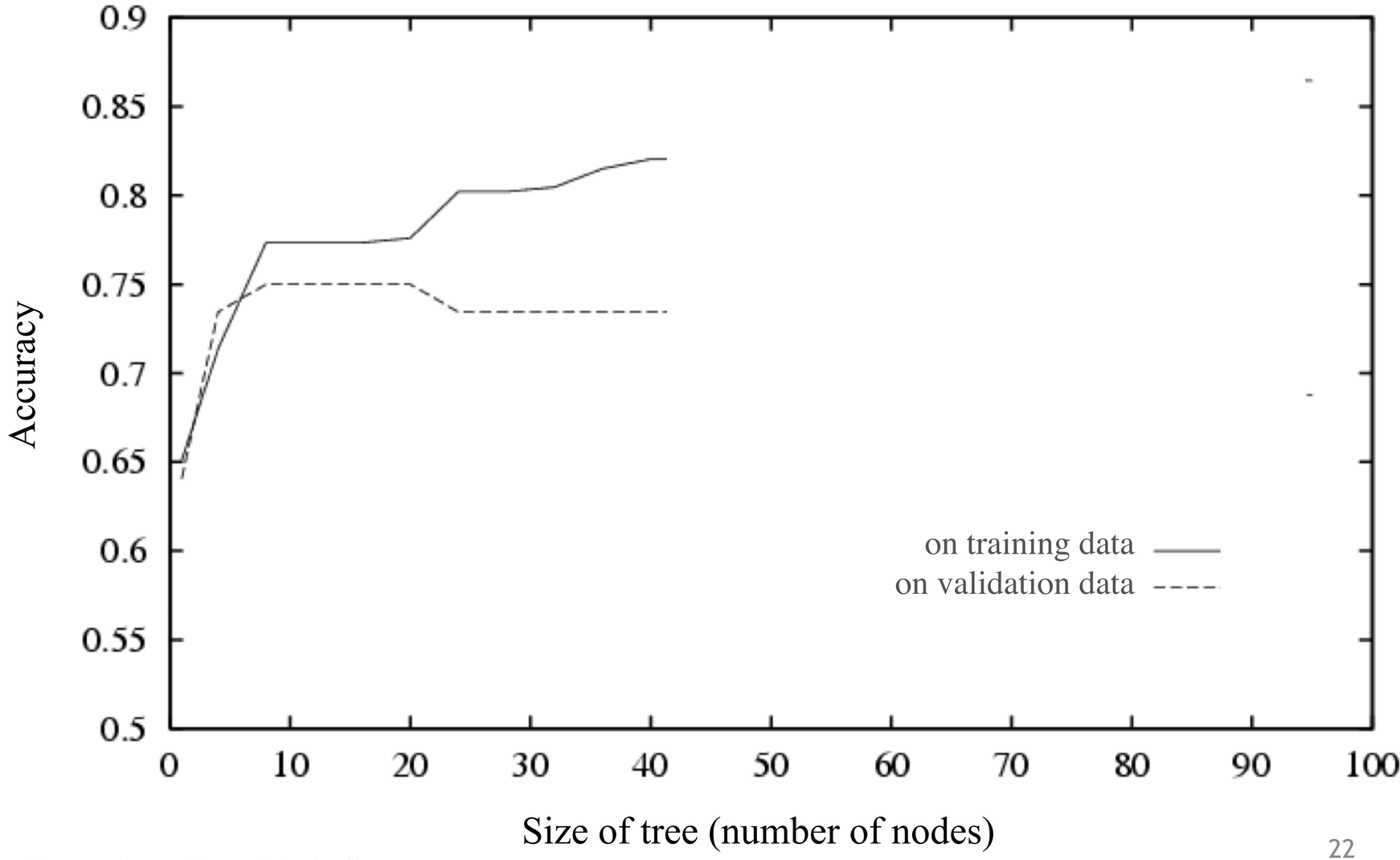


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

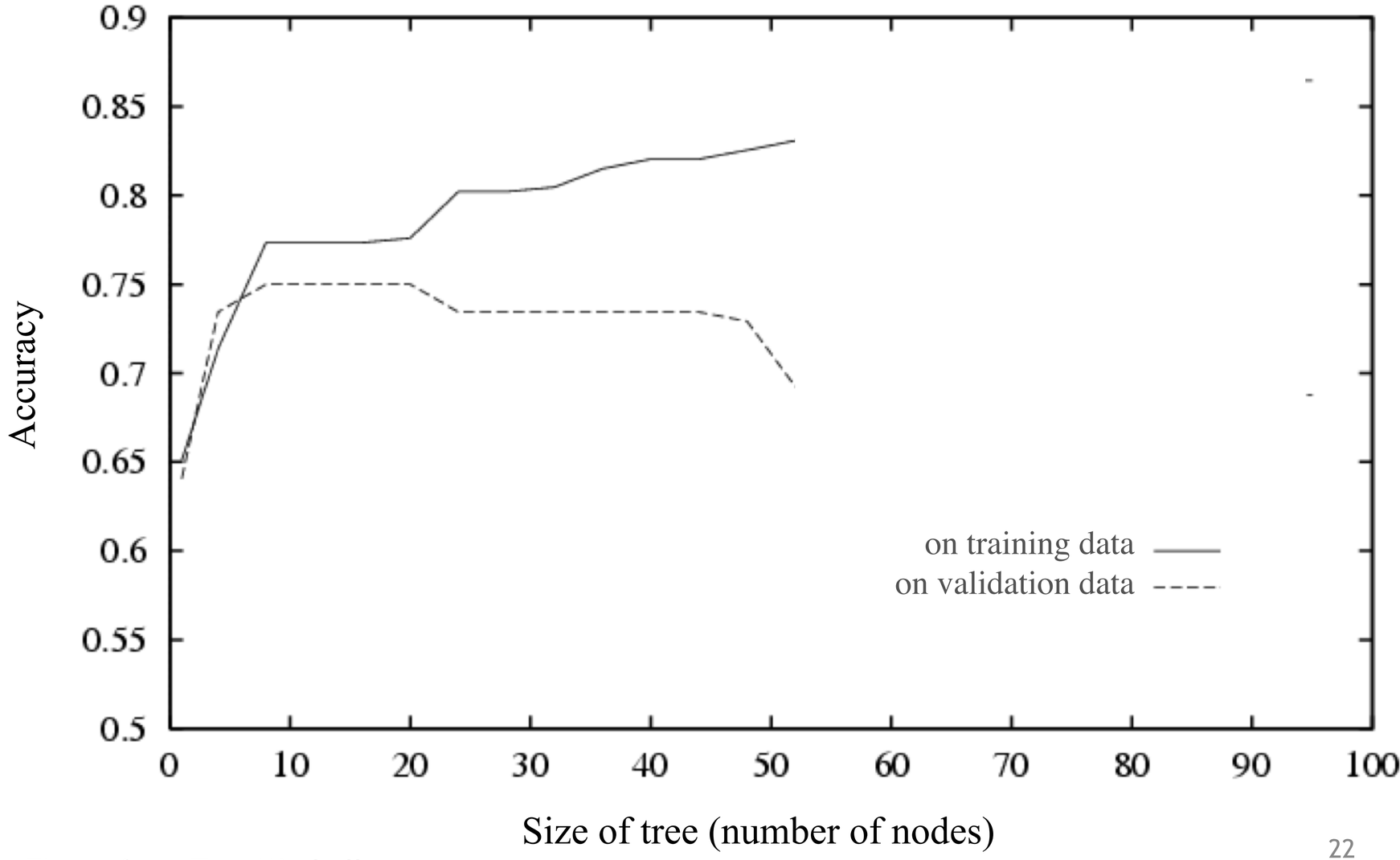


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

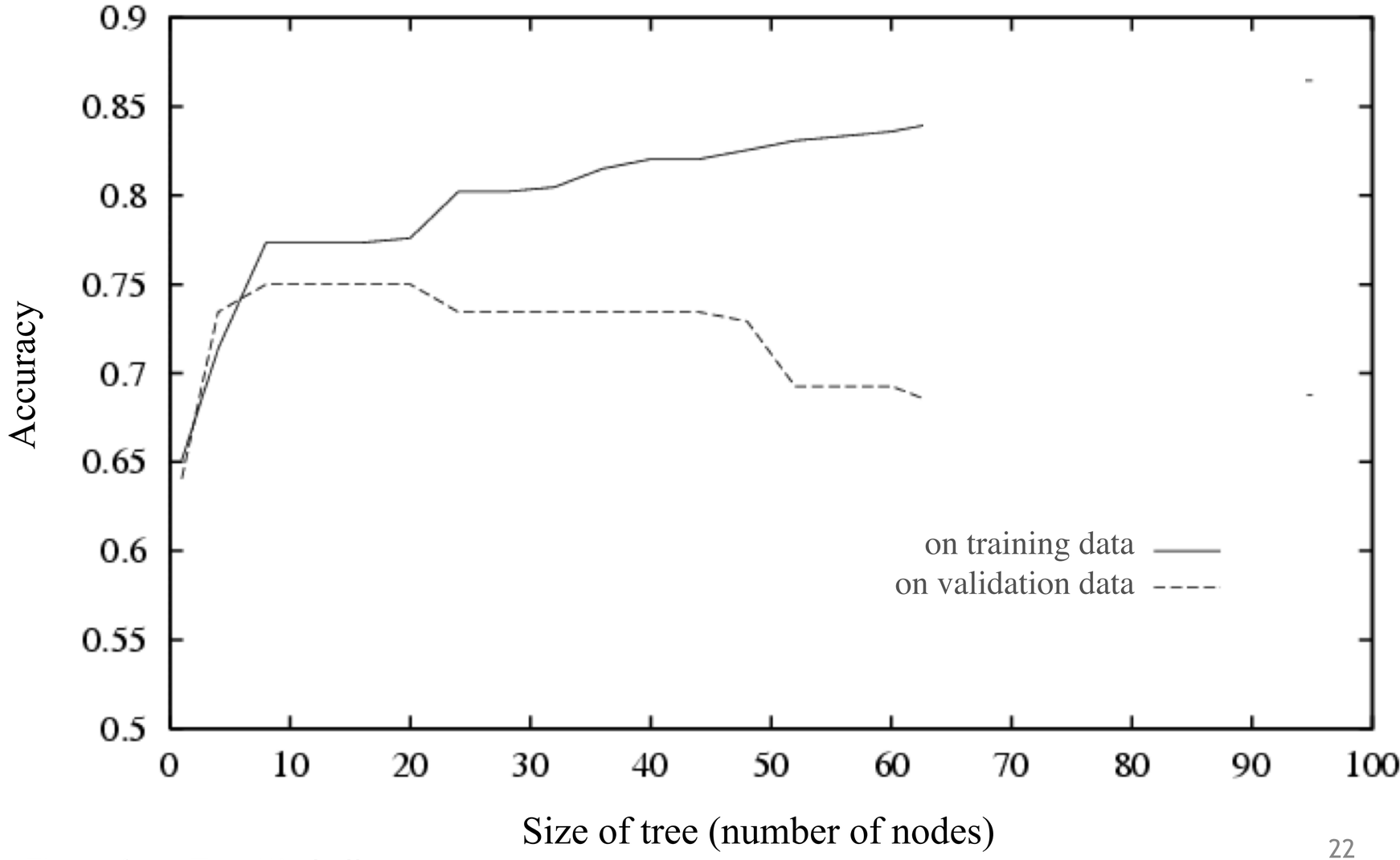


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

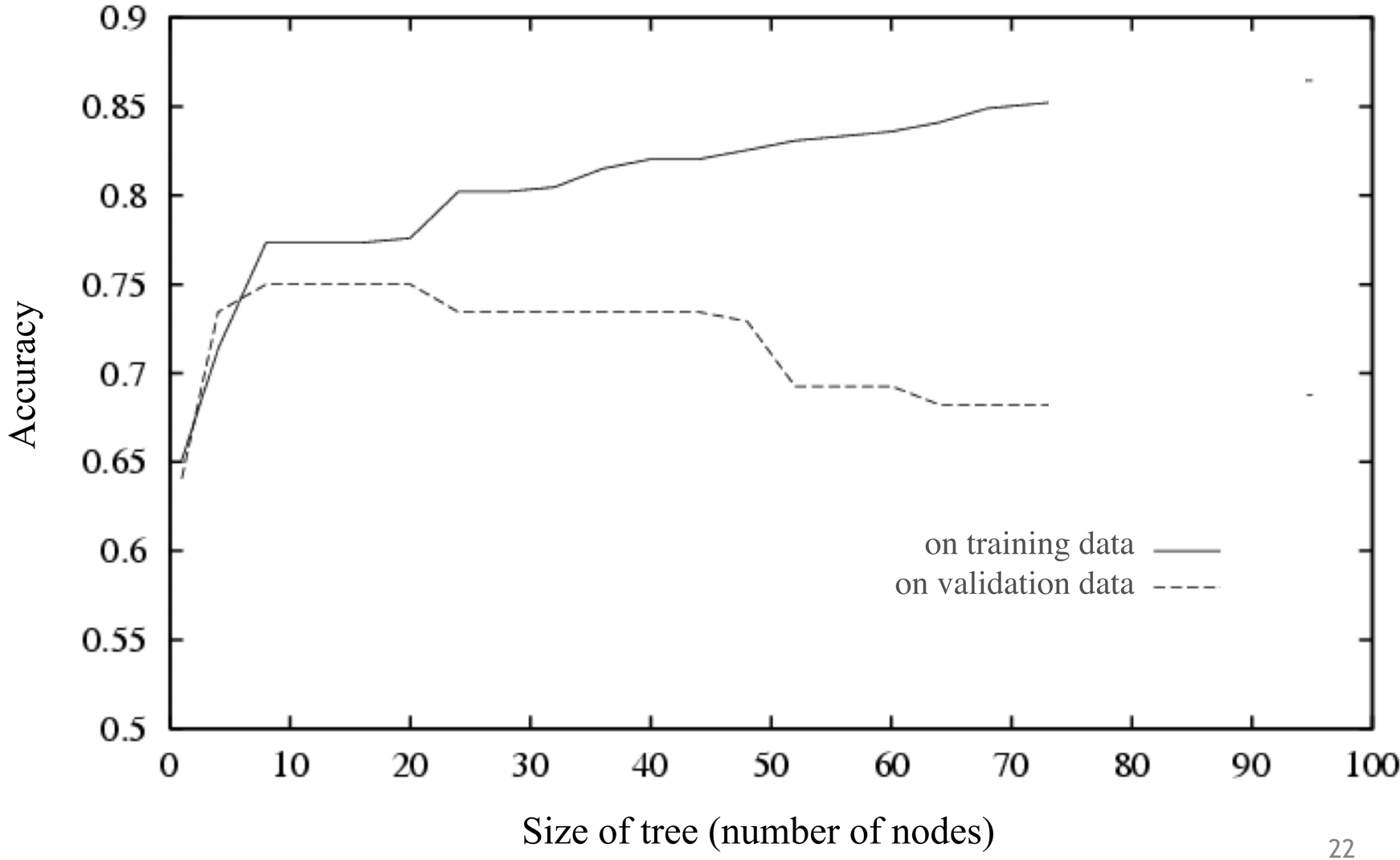


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

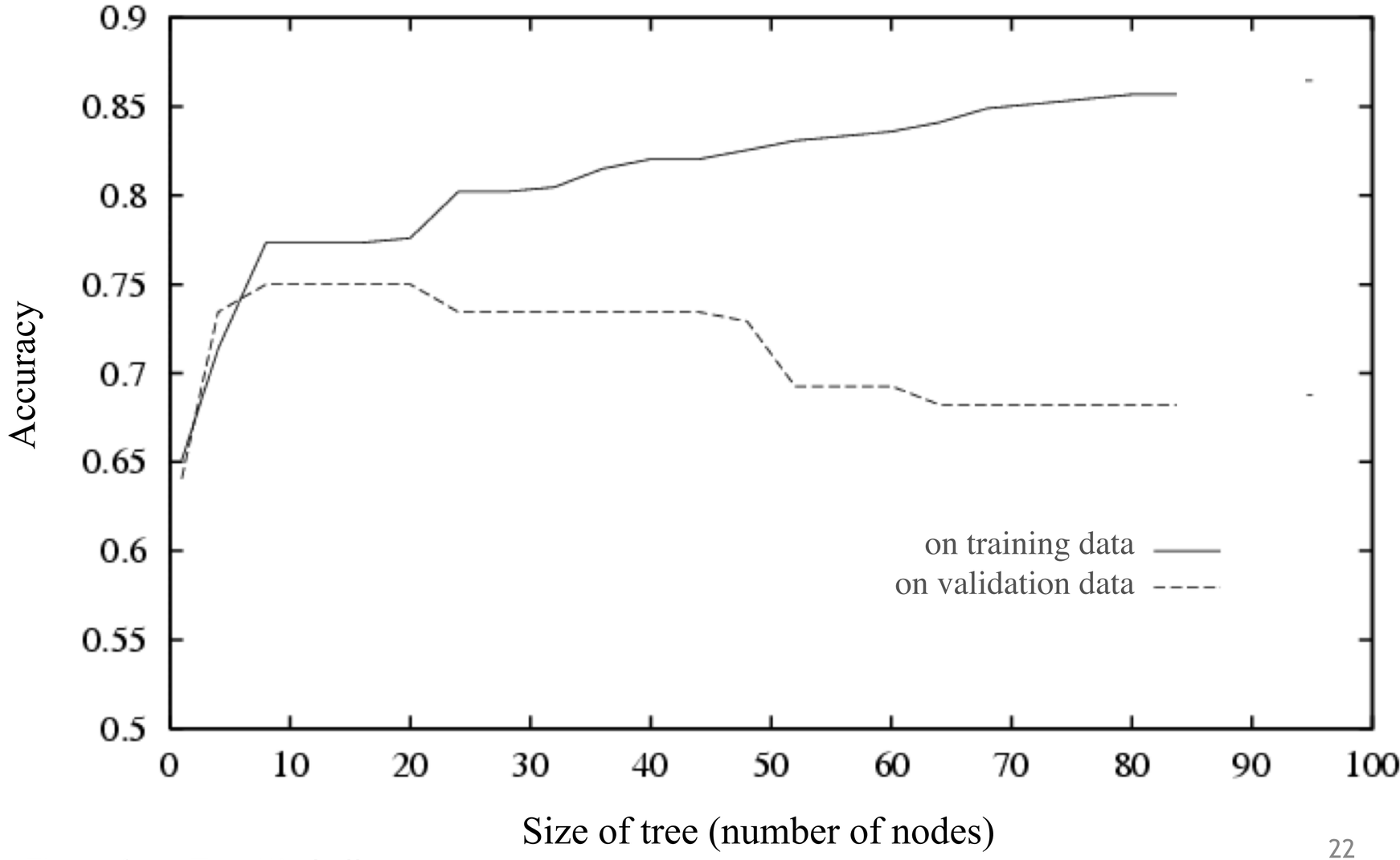


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

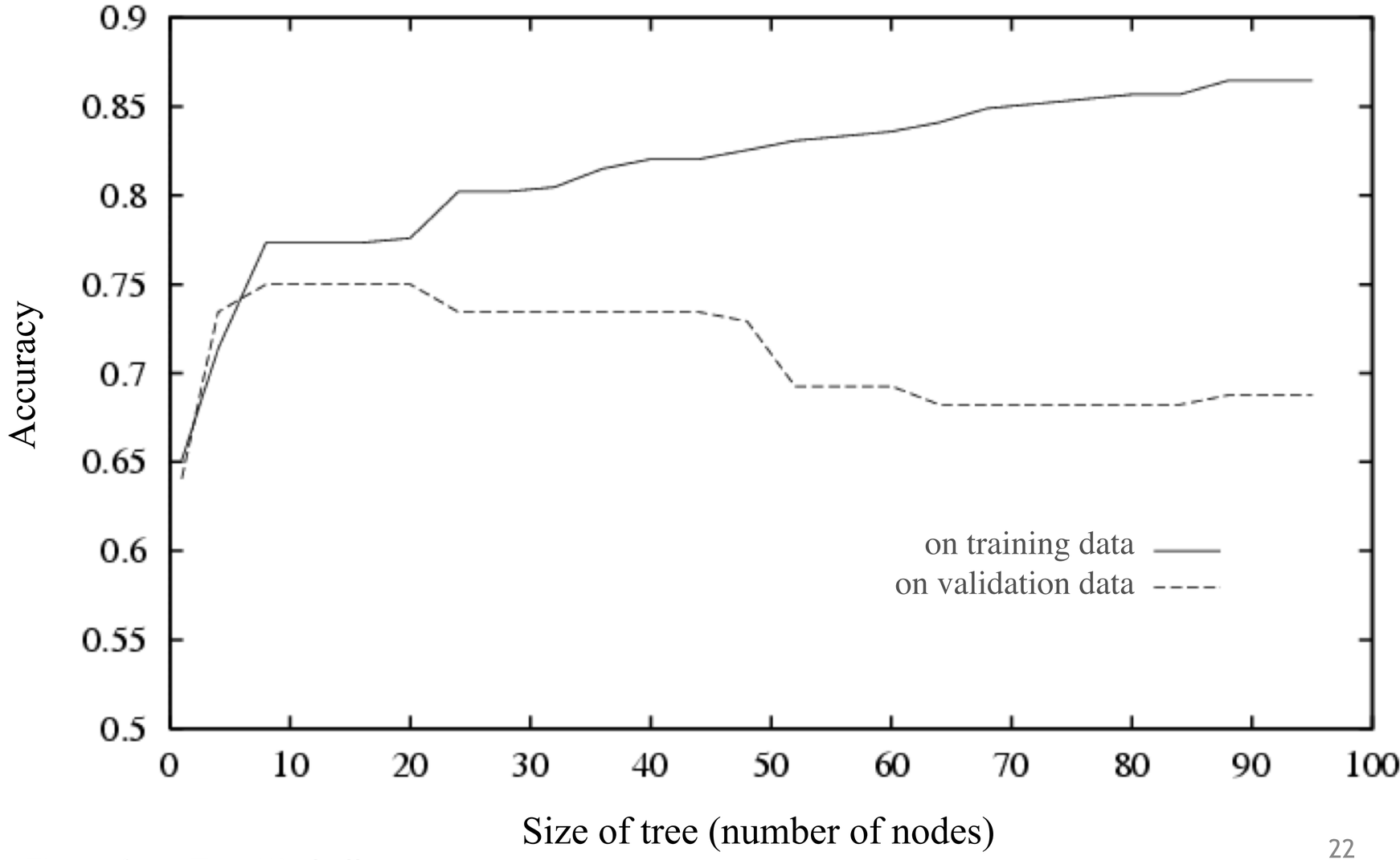


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

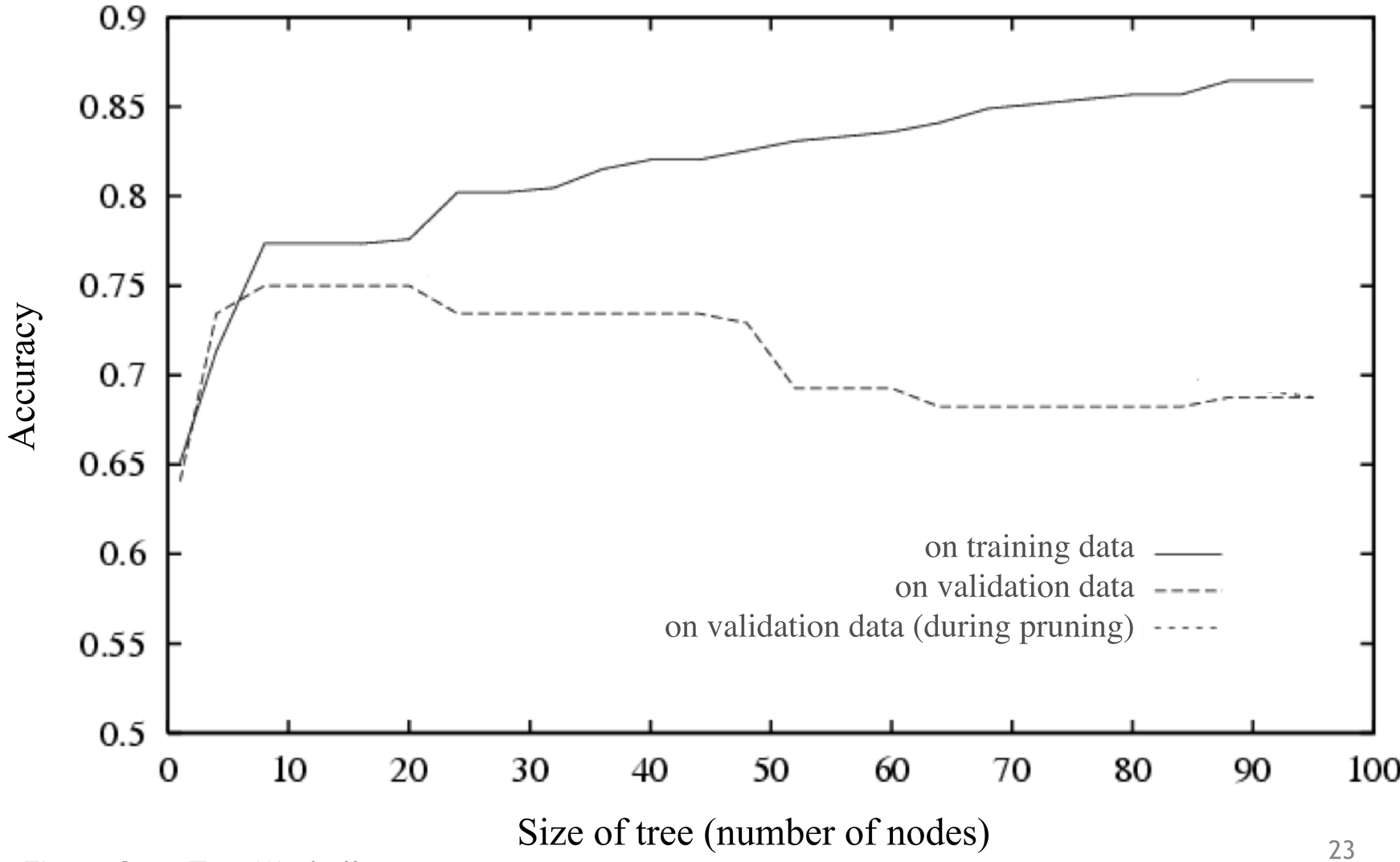


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

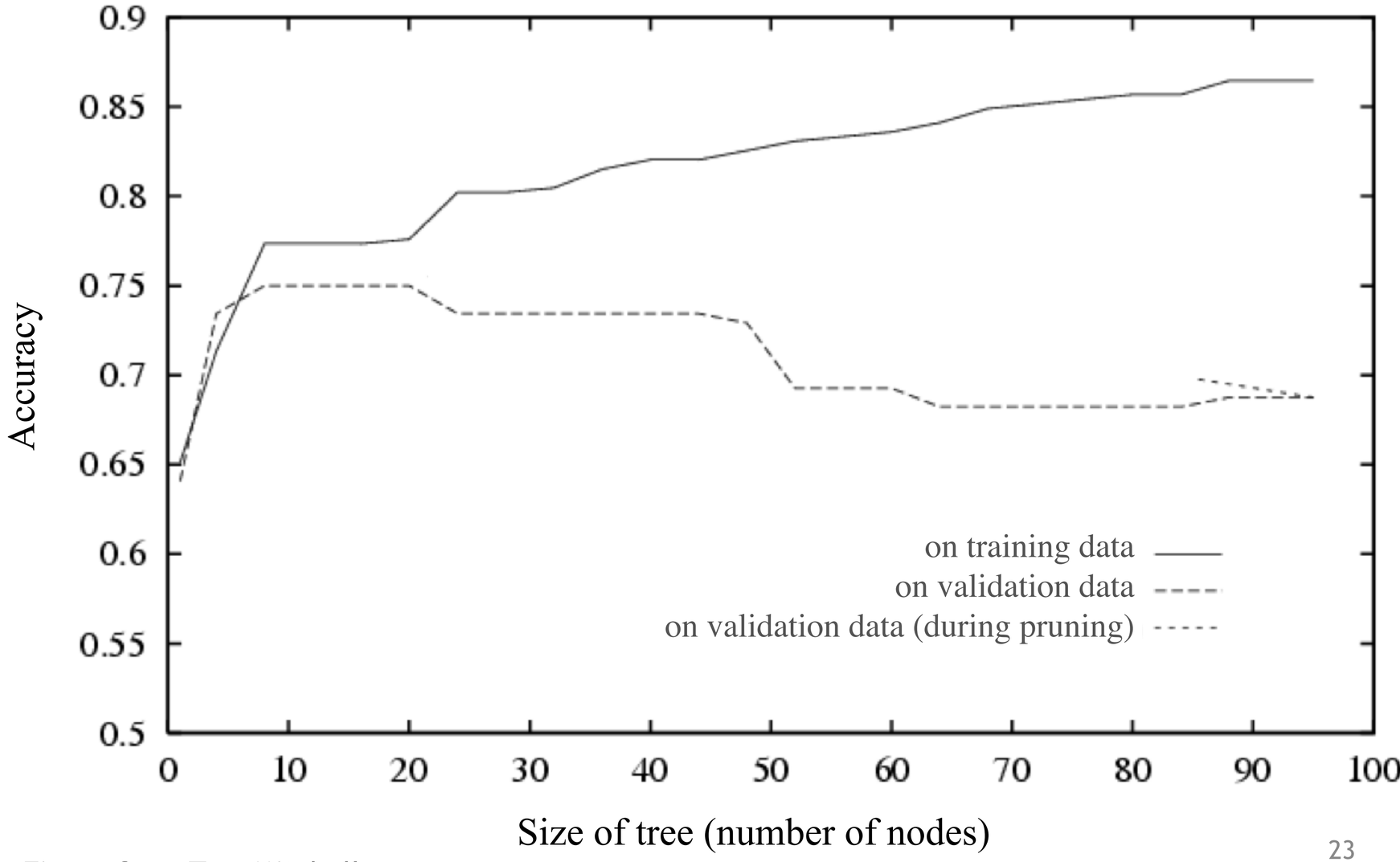


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

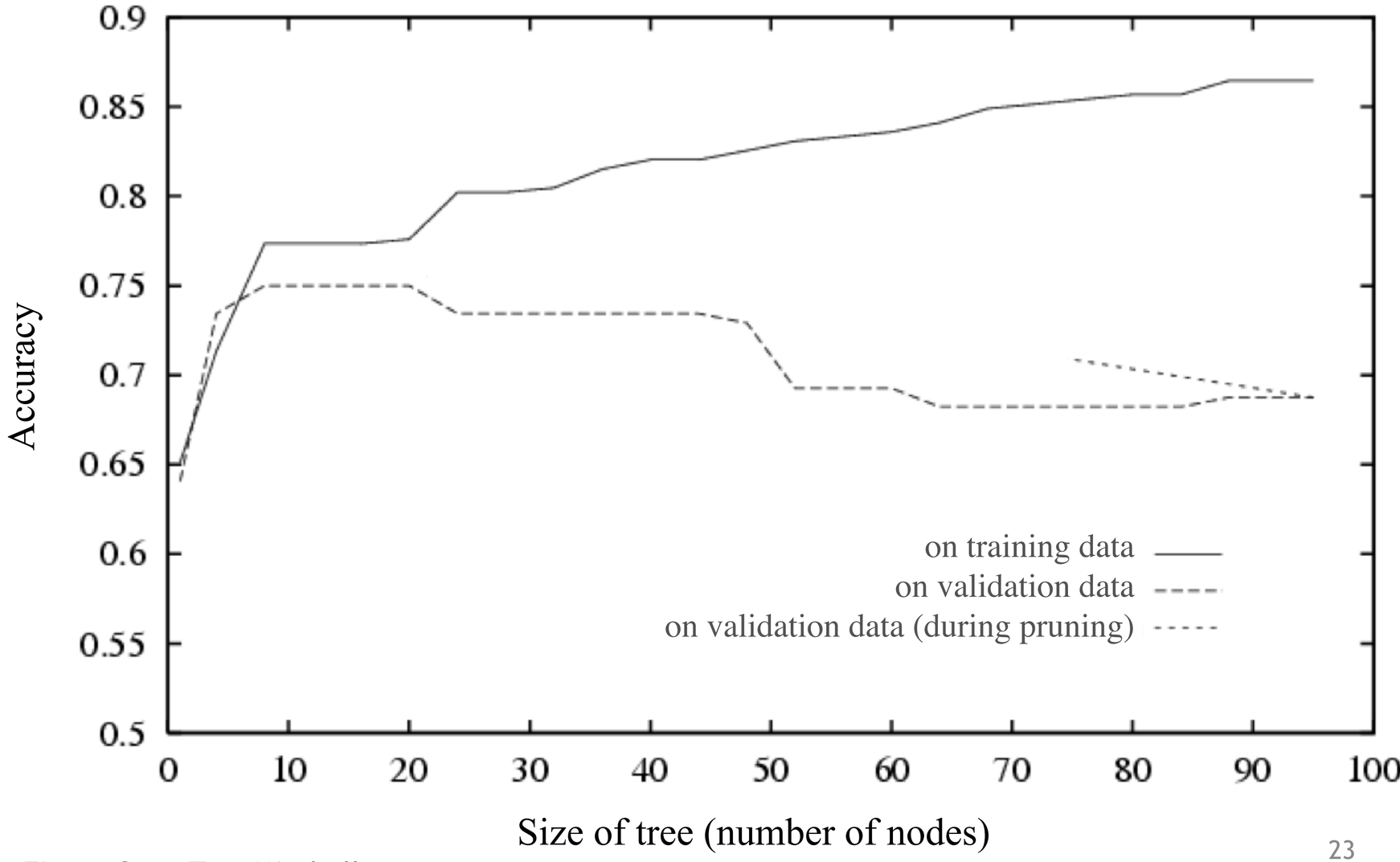


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

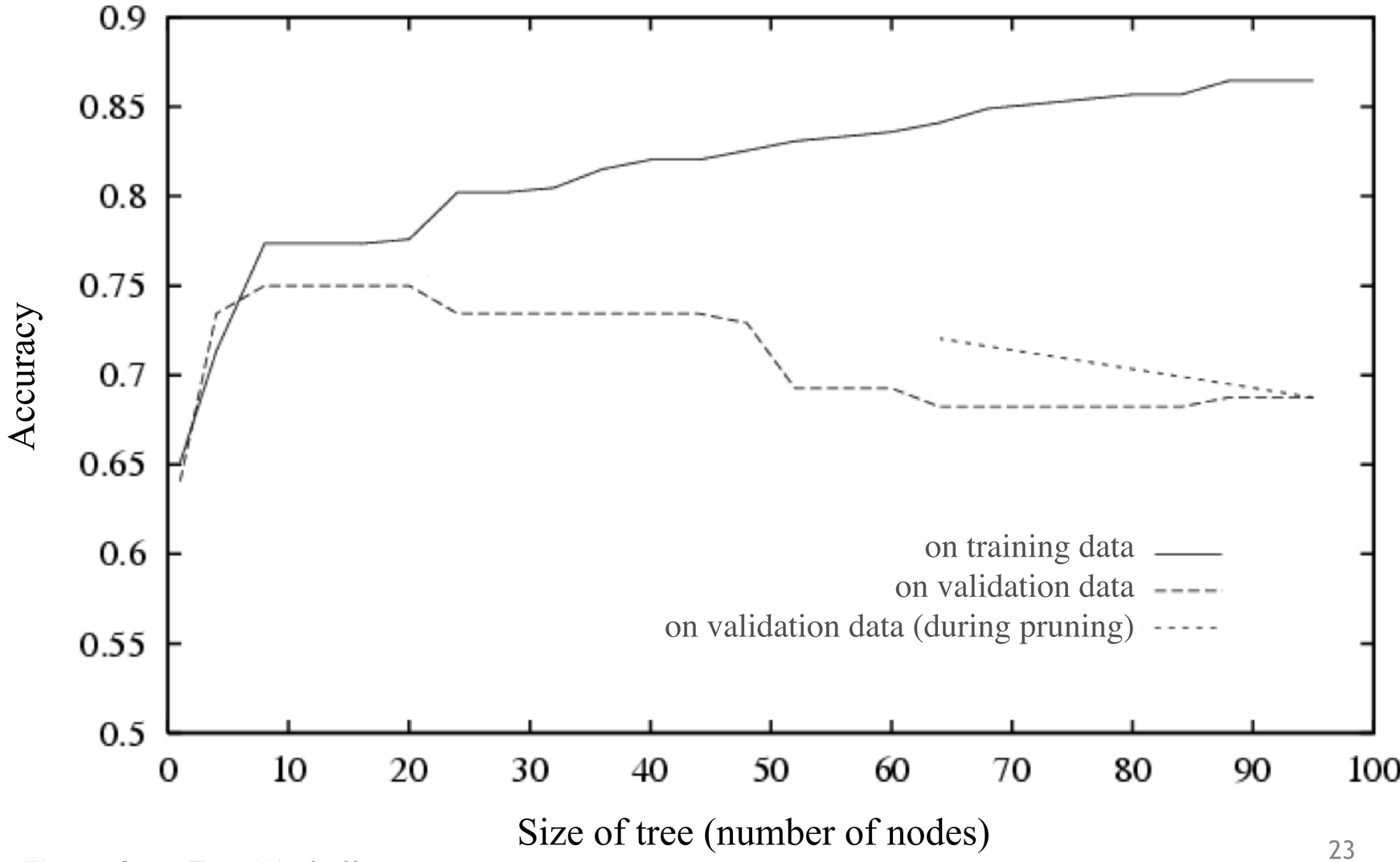


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

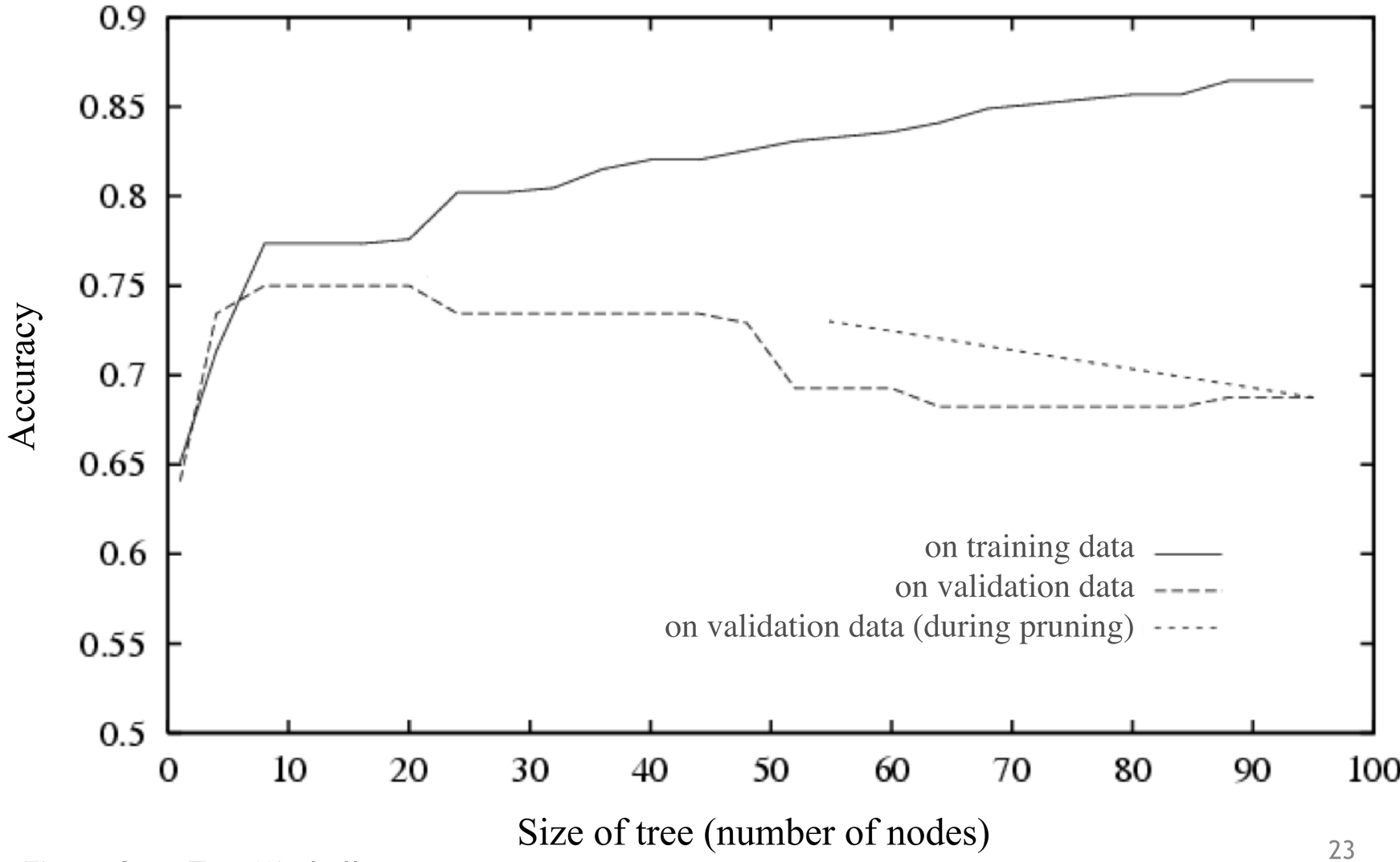


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

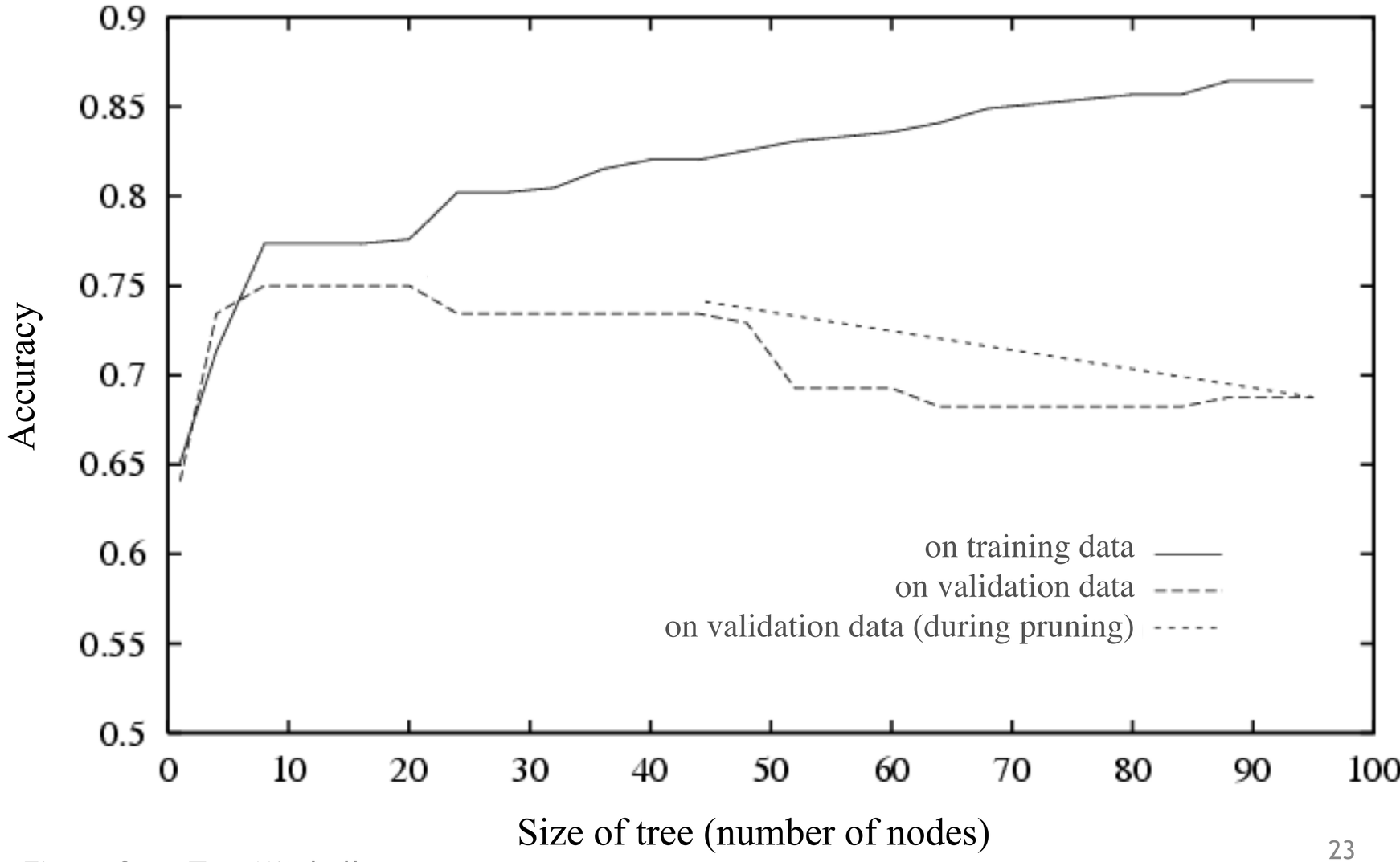


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

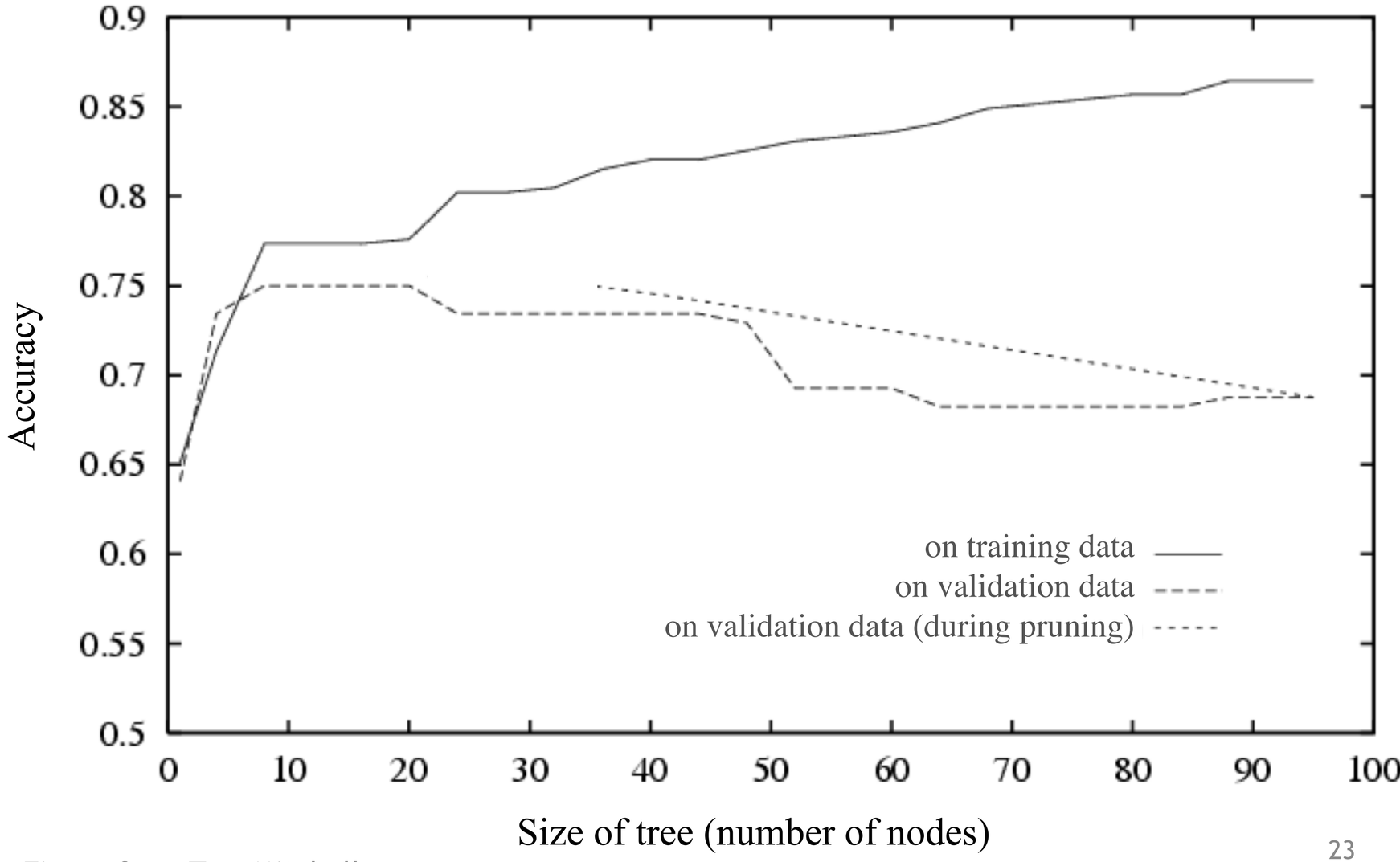


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

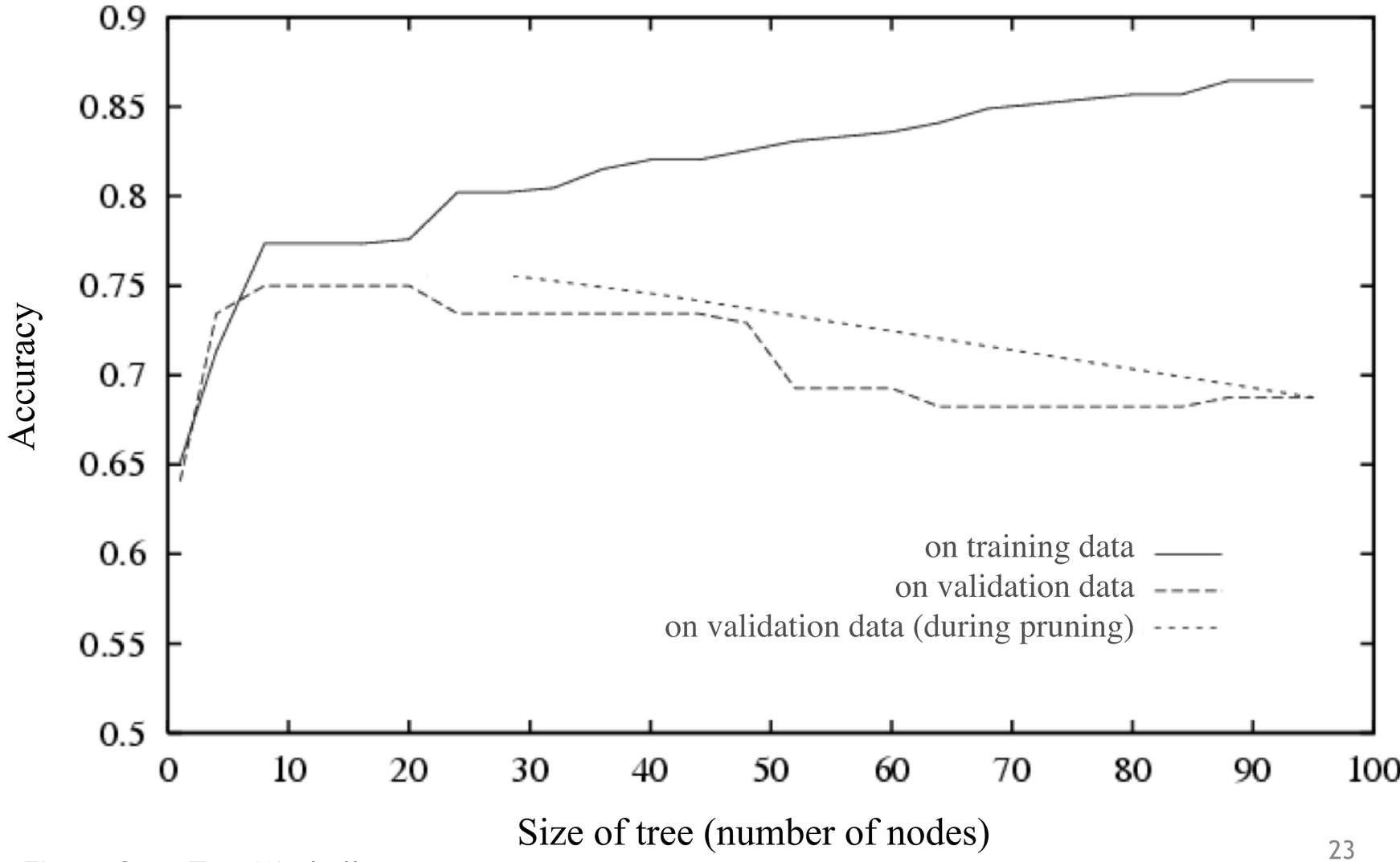


Figure from Tom Mitchell

Reduced Error Pruning

1. Split data in two: *training* dataset and *validation* dataset
2. Grow the full tree using the *training* dataset
3. Repeatedly prune the tree:
   - Evaluate each split using a *validation* dataset by comparing the validation error rate **with and without** that split
   - (Greedily) remove the split that most decreases the validation error rate
   - Stop if no split improves validation error, otherwise repeat

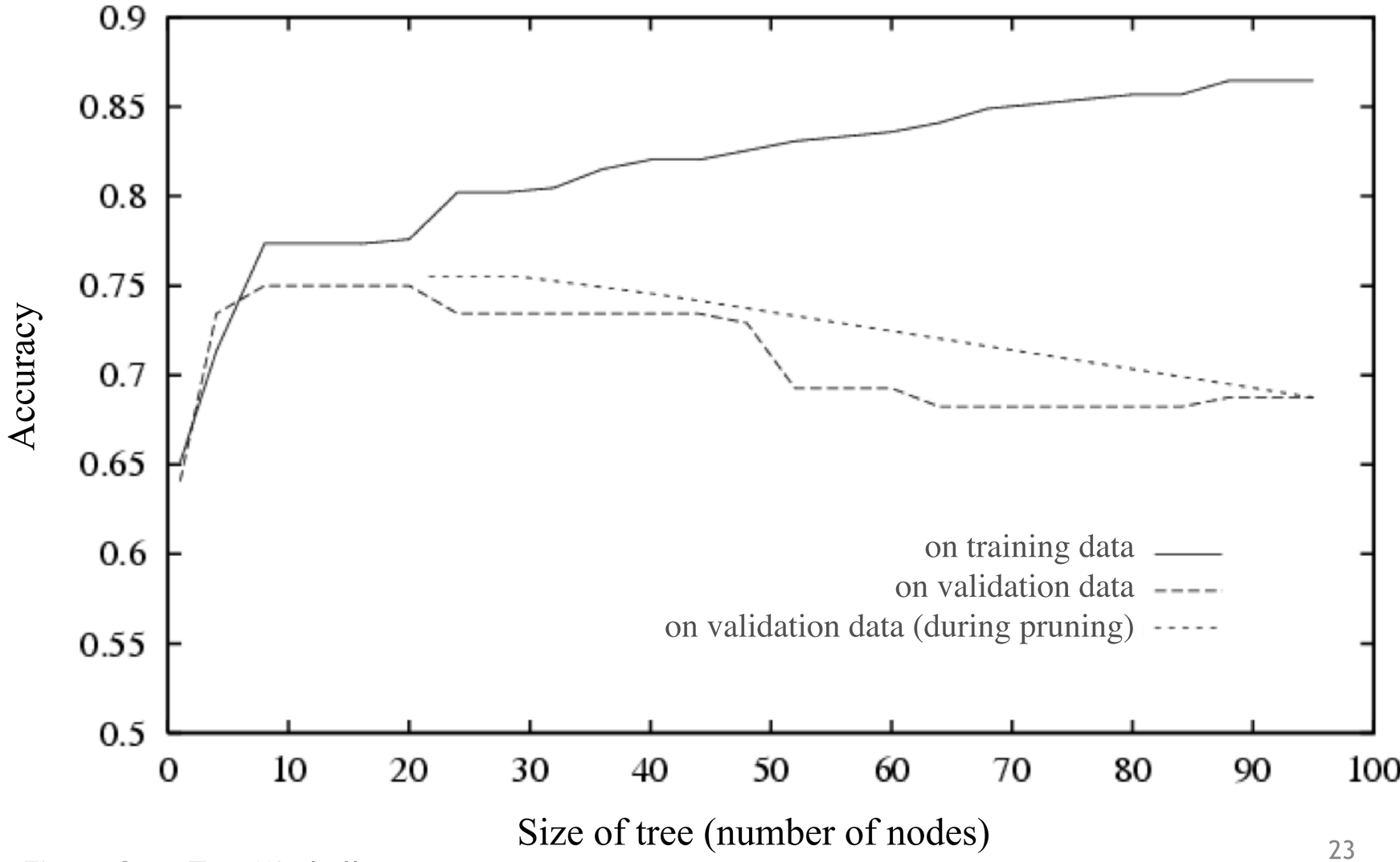
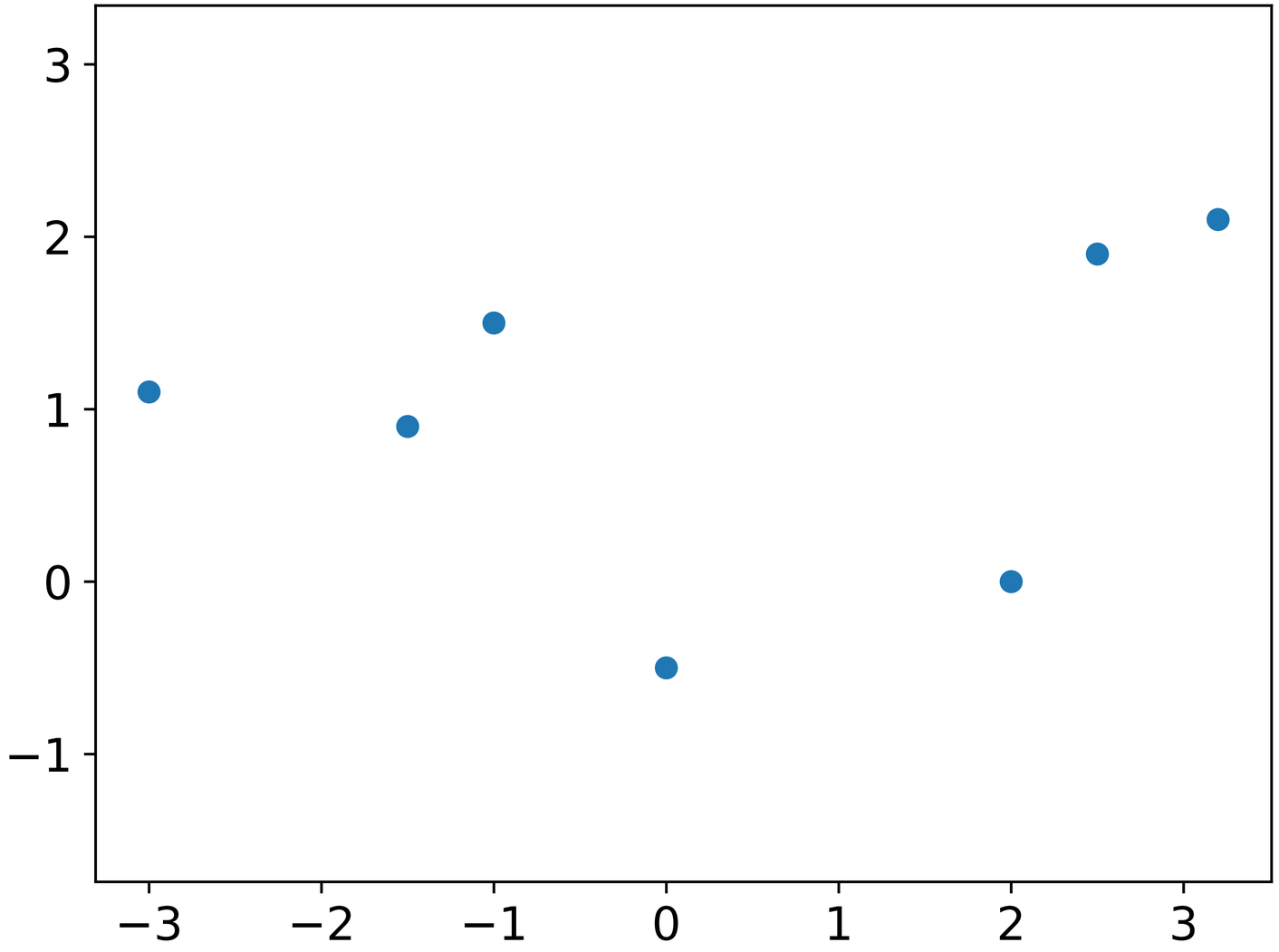


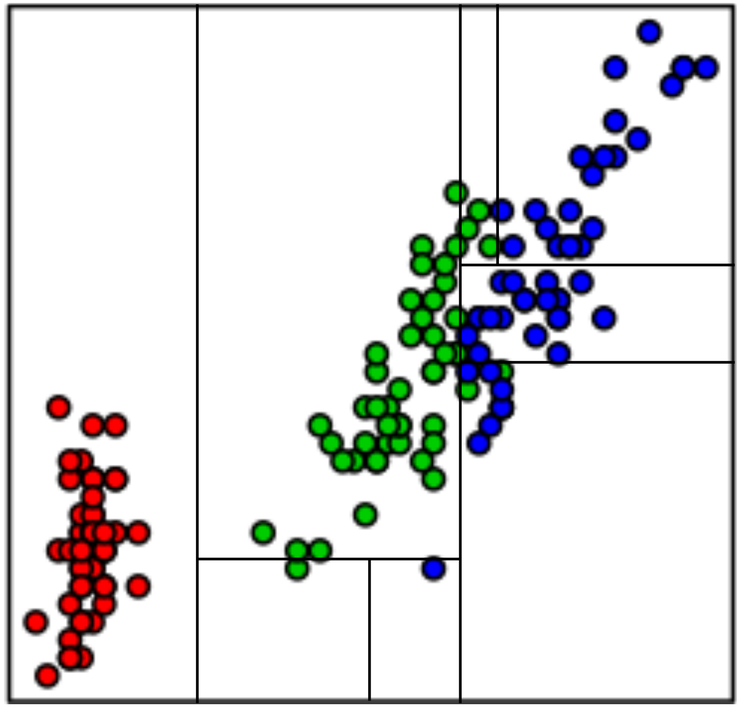
Figure from Tom Mitchell

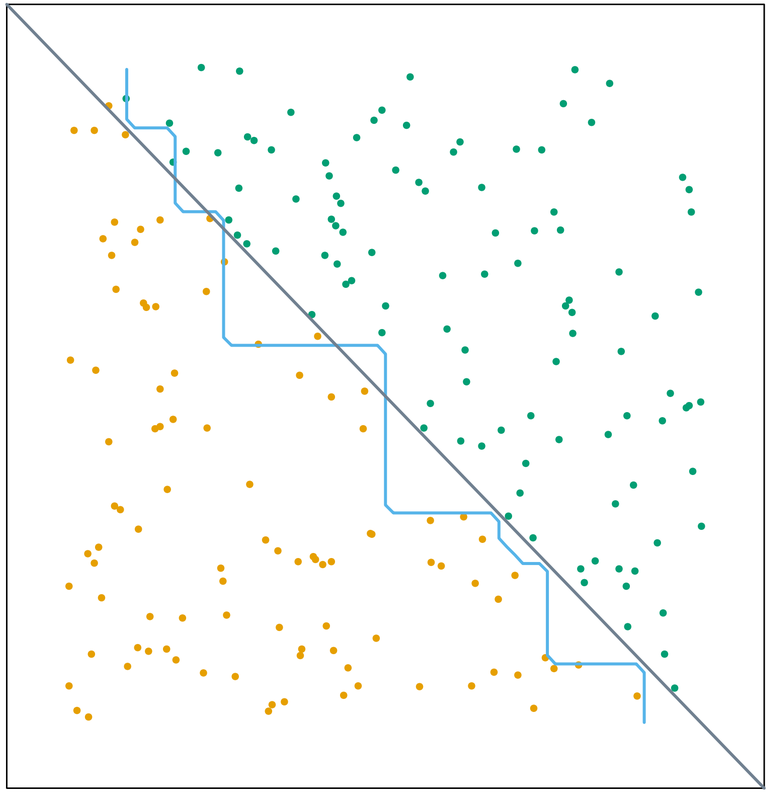
How to handle real outputs: regression tree



- In each leaf: predict mean
- Splitting criterion: prediction error $\sum_{\text{examples } i} (y_i - \hat{y}_i)^2$

Inductive bias of decision trees





- Divide space into nested regions
- Piecewise constant in regions
- Axis-parallel splits
- Decisions expressed as simple rules: sepal width ≤ 5.2 & sepal length ∈ [2.5, 4.0]

Learning objectives—decision trees

- What is a decision tree? How do we use it to predict?
- What is the inductive bias of a decision tree?
- How can we grow a decision tree?
- How do we use mutual information as a splitting criterion?
- How can we prune a decision tree? Why would we?
- What is a validation set, how does it differ from a test set, and why do we need one?